

Chapter 11. Rx: Reactive Extensions

The Reactive Extensions for .NET (usually shortened to *Rx*) are designed for working with asynchronous and event-based information sources. Rx provides services that help you orchestrate and synchronize the way your code reacts to data from these kinds of sources. We already saw how to define and subscribe to events in [Chapter 9](#), but Rx offers much more than these basic features. It provides an abstraction that has a steeper learning curve than events, but it comes with a powerful set of operators that makes it far easier to combine and manage multiple streams of events than is possible with the free-for-all that delegates and .NET events provide. Microsoft has also made an associated set of libraries called Reaqtor available that builds on the foundation of Rx to provide a framework for reliable, stateful, distributed, scalable, high-performance event processing in services. (The *q* in Reaqtor is a nod to the `IQueryable<T>` interface described in the preceding chapter.)

Rx's fundamental abstraction, `IObservable<T>`, represents a sequence of items, and its operators are defined as extension methods for this interface. This might sound a lot like LINQ to Objects, and there are similarities—not only does `IObservable<T>` have a lot in common with `IEnumerable<T>`, but Rx also supports almost all of the standard LINQ operators. If you are familiar with LINQ to Objects, you will also feel at home with Rx. The difference is that in Rx, sequences are less passive. Unlike `IEnumerable<T>`, Rx sources do not wait to be asked for their items, nor can the consumer of an Rx source demand to be given the next item. Instead, Rx uses a *push* model in which the source notifies its recipients when items are available.

For example, if you're writing an application that deals with live financial information, such as stock market price data, `IObservable<T>` is a much more natural model than `IEnumerable<T>`. Because Rx implements stan-

standard LINQ operators, you can write queries against a live source—you could narrow down the stream of events with a `where` clause or group them by stock symbol. Rx goes beyond standard LINQ, adding its own operators that take into account the temporal nature of a live event source. For example, you could write a query that provides data only for stocks that are changing price more frequently than some minimum rate.

Rx's push-oriented approach makes it a better match than

`IEnumerable<T>` for event-like sources. But why not just use events, or even plain delegates? Rx addresses four shortcomings of those alternatives. First, it defines a standard way for sources to report errors. Second, it is able to deliver items in a well-defined order, even in multithreaded scenarios involving numerous sources. Third, Rx provides a clear way to signal when there are no more items. Fourth, because a traditional event is represented by a special kind of member, not a normal object, there are significant limits on what you can do with an event—you can't pass a .NET event as an argument to a method, store it in a field, or offer it in a property.

You can do these things with a delegate, but that's not the same thing—delegates can handle events but cannot represent a source of them.

There's no way to write a method that subscribes to some .NET event that you pass as an argument, because you can't pass the actual event itself.

Rx fixes this by representing event sources as objects, instead of a special distinctive element of the type system that doesn't work like anything else.

We get all four of these features for free back in the world of

`IEnumerable<T>`, of course. A collection can throw an exception when its contents are being enumerated, but with callbacks, it's less obvious when and where to deliver exceptions. `IEnumerable<T>` makes consumers retrieve items one at a time, so the ordering is unambiguous, but with plain events and delegates, nothing enforces that. And `IEnumerable<T>` tells consumers when the end of the collection has been reached, but with a simple callback, it's not necessarily clear when you've had the last call.

`IObservable<T>` handles all of these eventualities, bringing the things we can take for granted with `IEnumerable<T>` into the world of events.

By providing a coherent abstraction that addresses these problems, Rx is able to bring all of the benefits of LINQ to event-driven scenarios. Rx does not replace events; I wouldn't have dedicated one-fifth of [Chapter 9](#) to them if it did. In fact, Rx can integrate with events. It can bridge between its own abstractions and several others, not just ordinary events but also `IEnumerable<T>` and various asynchronous programming models. Far from deprecating events, Rx raises their capabilities to a new level. It's considerably harder to get your head around Rx than events, but it offers much more power once you do.

Two interfaces form the heart of Rx. Sources that present items through this model implement `IObservable<T>`. Subscribers are required to supply an object that implements `IObserver<T>`. These two interfaces are built into .NET. The other parts of Rx are in the `System.Reactive` NuGet package. That package is an [open source project](#) that was originally written by Microsoft (hence the `System` namespace), but which is now a community-maintained .NET Foundation project.

Fundamental Interfaces

There are two essential types in Rx: the `IObservable<T>` and `IObserver<T>` interfaces. These are important enough to be in the `System` namespace. [Example 11-1](#) shows their definitions.

Example 11-1. `IObservable<T>` and `IObserver<T>`

```
public interface IObservable<out T>
{
    IDisposable Subscribe(IObserver<T> observer);
}

public interface IObserver<in T>
{
    void OnCompleted();
    void OnError(Exception error);
    void OnNext(T value);
}
```

The fundamental abstraction in Rx, `IObservable<T>`, is implemented by event sources. Instead of using the `event` keyword, it models events as a sequence of items. An `IObservable<T>` can have any number of subscribers, and it provides them with items as and when it's ready to.

As you can see from the `out` keyword, the type argument for `IObservable<T>` is covariant, meaning that if you have a type `Base` that is the base type of another type `Derived`, then just as you can pass a `Derived` to any method expecting a `Base`, you can pass an `IObservable<Derived>` to anything expecting an `IObservable<Base>`. It makes sense intuitively to see the `out` keyword here, because like `IEnumerable<T>`, this is a source of information—items come out of it. Conversely, items go into a subscriber's `IObserver<T>` implementation, so that has the `in` keyword, which denotes contravariance—you can pass an `IObserver<Base>` to anything expecting an `IObserver<Derived>`. (I described variance in [Chapter 6](#).)

We can subscribe to a source by passing an implementation of `IObserver<T>` to the `Subscribe` method. The source will invoke `OnNext` when it wants to report events. Rx has a basic rule that the source is required to wait until `OnNext` returns before either calling `OnNext` again, or calling `OnError` or `OnComplete`. This rule keeps things simple for observers: even in multithreaded applications, any single observer will only ever have to deal with one thing at a time. (More subtly, it may also require a source to detect re-entrancy: if the observer's `OnNext` performs some action that indirectly causes the source to emit another item, the source is not allowed to make a recursive call into the observer. It has to wait until the `OnNext` in progress returns. This can make things complex for sources, but as you'll see, the Rx libraries can help with that.) The source can call `OnCompleted` to indicate that there will be no further activity, and if it wants to report an error, it can call `OnError`. Both `OnCompleted` and `OnError` indicate the end of the stream, so another basic Rx rule is that an observable should not call any further methods on the observer after that.

WARNING

You will not necessarily get an exception immediately if you break these rules. If you use the NuGet `System.Reactive` library to help implement and consume these interfaces, there are certain circumstances in which it can detect this kind of mistake. But in general it is the responsibility of code calling the `IObserver<T>` methods to stick to the rules.

There's a visual convention for representing Rx activity. It's sometimes called a *marble diagram*, because it consists mainly of small circles that look a bit like marbles. [Figure 11-1](#) uses this convention to represent two sequences of events. The horizontal lines represent subscriptions to sources, with the vertical bar on the left indicating the start of the subscription, and the horizontal position indicating when something occurred (with elapsed time increasing from left to right). The circles indicate calls to `OnNext` (i.e., events being reported by the source). An arrow on the righthand end indicates that the subscription was still active by the end of the time the diagram represents. A vertical bar on the right indicates the end of the subscription—either due to a call to `OnError` or `OnCompleted` or because the subscriber unsubscribed.

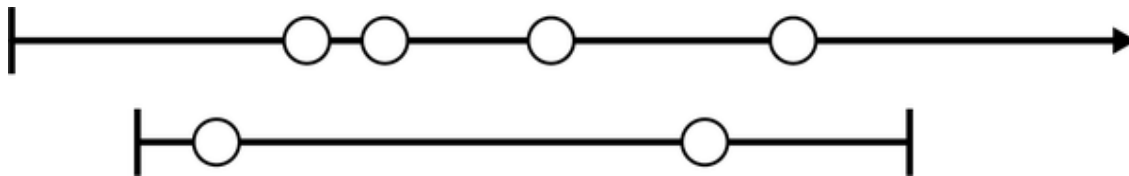


Figure 11-1. Simple marble diagram

When you call `Subscribe` on an observable, it returns an object that implements `IDisposable`, which provides a way to unsubscribe. If you call `Dispose`, the observable will not deliver any more notifications to your observer. This can be more convenient than the mechanism for unsubscribing from an event. To unsubscribe from an event, you must pass in an equivalent delegate to the one you used for subscription. If you're using anonymous methods, that can be surprisingly awkward, because often the only way to do that is to keep hold of a reference to the original delegate. With Rx, any subscription to a source is represented as an `IDisposable`, making it easier to handle in a uniform way. In fact, you often do not need to unsubscribe anyway—this is necessary only if you

want to stop receiving notifications before the source completes (making this an example of something that is relatively unusual in .NET: optional disposability).

IObserver<T>

As you'll see, in practice we often don't call a source's `Subscribe` method directly, nor do we usually need to implement `IObserver<T>` ourselves. Instead, it's common to use one of the delegate-based extension methods that Rx provides and that attaches an Rx-supplied implementation. However, those extension methods are not part of Rx's fundamental types, and they are in the optional `System.Reactive` NuGet package, not the .NET runtime libraries. So for now I'll show what you'd need to write if these interfaces are all you've got. [Example 11-2](#) shows a simple but complete observer.

Example 11-2. Simple `IObserver<T>` implementation

```
class MySubscriber<T> : IObserver<T>
{
    public void OnNext(T value) => Console.WriteLine("Received: " + value);
    public void OnCompleted() => Console.WriteLine("Complete");
    public void OnError(Exception ex) => Console.WriteLine("Error: " + ex);
}
```

Observers can often be very simple thanks to the guarantees that Rx sources (i.e., implementations of `IObservable<T>`) are required to make about how they call an observer's methods. Observers can safely assume that the calls will happen in a certain order: `OnNext` is called for each item that the source provides, and once either `OnCompleted` or `OnError` is called, there will be no further calls to any of the three methods.

Furthermore, the observer can count on the fact that calls are not allowed to overlap—when an observable source calls one of our observer's methods, it must wait for that method to return before calling again. When we write an observer, we don't need to worry about multithreaded calls, and even in a single-threaded world, it's not our problem to detect and prevent re-entrant calls—that's the source's job.

This makes life simple for the observer. Rx always provides events as a sequence. So, although `IObservable<T>` may look like the simpler interface, having just one method, it's the more demanding one to implement. As you'll see later, it's usually easiest to let the Rx libraries implement this for you, but it's still important to know how observable sources work, so I'll implement it by hand to begin with.

IObservable<T>

Rx makes a distinction between *hot* and *cold* observable sources. A hot observable produces each value as and when something of interest happens, and if no subscribers are attached at that moment, that value will be lost. A hot observable typically represents something live, such as mouse input, keypresses, or data reported by a sensor, which is why the values it produces are independent of how many subscribers, if any, are attached. Hot sources typically have broadcast-like behavior—they send each item to all of their subscribers. These can be the more complex kind of source to implement, so I'll discuss cold sources first.

Implementing cold sources

Whereas hot sources report items as and when they want to, cold observables work differently. They start pushing values when an observer subscribes, and they provide values to each subscriber separately, rather than broadcasting. This means that a subscriber won't miss anything by being too late, because the source starts providing items when you subscribe. [Example 11-3](#) shows a very simple cold source.

Example 11-3. A simple cold observable source

```
public class SimpleColdSource : IObservable<string>
{
    public IDisposable Subscribe(IObserver<string> observer)
    {
        observer.OnNext("Hello,");
        observer.OnNext("World!");
        observer.OnCompleted();
        return EmptyDisposable.Instance;
    }
}
```



```

    }

    private class EmptyDisposable : IDisposable
    {
        public readonly static EmptyDisposable Instance = new();
        public void Dispose() { }
    }
}

```

The moment an observer subscribes, this source will provide two values, the strings "Hello," and "World!", and will then indicate the end of the sequence by calling `OnCompleted`. It does all that inside `Subscribe`, so this doesn't really look like a subscription—the sequence is already over by the time `Subscribe` returns, so there's nothing meaningful to do to support unsubscription. That's why this returns a trivial implementation of `IDisposable`. (I've chosen an extremely simple example so I can show the basics. Real sources will be more complex.)

To show this in action, we need to create an instance of `SimpleColdSource`, and also an instance of my observer class from [Example 11-2](#), and use that to subscribe to the source, as [Example 11-4](#) does.

Example 11-4. Attaching an observer to an observable

```

var source = new SimpleColdSource();
var sub = new MySubscriber<string>();
source.Subscribe(sub);

```

Predictably, this produces the following output:

```

Received: Hello,
Received: World!
Complete

```

In general, a cold observer will have access to some underlying source of information, which it can push to a subscriber on demand. In [Example 11-3](#), that “source” was just two hardcoded values. [Example 11-5](#)

shows a slightly more interesting cold observable, which reads the lines out of a file and provides them to a subscriber.

Example 11-5. A cold observable representing a file's contents

```
public class FilePusher : IObservable<string>
{
    private readonly string _path;
    public FilePusher(string path)
    {
        _path = path;
    }

    public IDisposable Subscribe(IObserver<string> observer)
    {
        using (var sr = new StreamReader(_path))
        {
            while (sr.ReadLine() is string line) // Repeats until null returned
            {
                observer.OnNext(line);
            }
        }
        observer.OnCompleted();
        return EmptyDisposable.Instance;
    }

    private class EmptyDisposable : IDisposable
    {
        public static EmptyDisposable Instance = new();
        public void Dispose() { }
    }
}
```

As before, this does not represent a live source of events, and it leaps into action only when something subscribes, but it's a little more interesting than [Example 11-3](#). This calls into the observer as and when it retrieves each line from a file, so although the point at which it starts doing its work is determined by the subscriber, this source is in control of the rate at which it provides values. Just like [Example 11-3](#), this delivers all the items to the observer on the caller's thread inside the call to `Subscribe`,

but it would be a relatively small conceptual leap from [Example 11-5](#) to one in which the code reading from the file either ran on a separate thread or used asynchronous techniques (such as those described in [Chapter 17](#)), thus enabling `Subscribe` to return before the work is complete (at which point you'd need to write a more interesting `IDisposable` implementation to enable callers to unsubscribe). This would still be a cold source, because it represents some underlying set of data that it can enumerate from the start for the benefit of each individual subscriber.

[Example 11-5](#) is not quite complete—it fails to handle errors that occur while reading from the file. We need to catch these and call the observer's `OnError` method. Unfortunately, it's not quite as simple as wrapping the whole loop in a `try` block, because that would also catch exceptions that emerged from the observer's `OnNext` method. If that throws an exception, we should allow it to carry on up the stack—we should handle only exceptions that emerge from the places we expect in our code. Unfortunately, this rather complicates the code. [Example 11-6](#) puts all the code that uses `FileStream` inside a `try` block but will allow any exceptions thrown by the observer to propagate up the stack, because it's not up to us to handle those.

Example 11-6. Handling filesystem errors but not observer errors

```
public IDisposable Subscribe(IObserver<string> observer)
{
    StreamReader? sr = null;
    string? line = null;

    try
    {
        while (true)
        {
            try
            {
                sr ??= new StreamReader(_path);
                line = sr.ReadLine();
            }
            catch (IOException ex)
            {
```

```

        observer.OnError(ex);
        break;
    }

    if (line is not null)
    {
        observer.OnNext(line);
    }
    else
    {
        observer.OnCompleted();
        break;
    }
}
}
finally
{
    sr?.Dispose();
}
return EmptyDisposable.Instance;
}

```

If I/O exceptions occur while reading from the file, this reports them to the observer's `OnError` method—so this source uses all three of the `IObserver<T>` methods.

Implementing hot sources

Hot sources notify all current subscribers of values as they become available. This means that any hot observable must keep track of which observers are currently subscribed. Subscription and notification are separated out with hot sources in a way that they usually aren't with cold ones.

[Example 11-7](#) is an observable source that reports a single item for each keypress, and it's a particularly simple source as hot ones go. It's single-threaded, so it doesn't need to do anything special to avoid overlapping calls. It doesn't report errors, so it never needs to call observers' `OnError` methods. And it never stops, so it doesn't need to call `OnCompleted` either. Even so, it's quite involved. (Things will get much simpler once I in-

roduce the Rx library support—this example is relatively complex because for now, I’m sticking with just the two fundamental interfaces.)

Example 11-7. IObservable<T> for monitoring keypresses

```
public class KeyWatcher : IObservable<char>
{
    private readonly List<Subscription> _subscriptions = new();

    public IDisposable Subscribe(IObserver<char> observer)
    {
        var sub = new Subscription(this, observer);
        _subscriptions.Add(sub);
        return sub;
    }

    public void Run()
    {
        while (true)
        {
            // Passing true here stops the console from showing the character
            char c = Console.ReadKey(true).KeyChar;

            // ToArray duplicates the list, enabling us to iterate over a
            // snapshot of our subscribers. This avoids errors when an
            // observer unsubscribes from inside its OnNext method.
            foreach (Subscription sub in _subscriptions.ToArray())
            {
                sub.Observer.OnNext(c);
            }
        }
    }

    private class Subscription(KeyWatcher parent, IObserver<char> observer)
        : IDisposable
    {
        private KeyWatcher? _parent = parent;

        public IObserver<char> Observer { get; } = observer;

        public void Dispose()
        {
            _parent?._subscriptions.Remove(this);
        }
    }
}
```

```

        if (_parent is not null)
        {
            _parent._subscriptions.Remove(this);
            _parent = null;
        }
    }
}

```

This defines a nested class called `Subscription` to keep track of each observer that subscribes, and this also provides the implementation of `IDisposable` that our `Subscribe` method is required to return. The observable creates a new instance of this nested class and adds it to a list of current subscribers during `Subscribe`, and then if `Dispose` is called, it removes itself from that list.

As a general rule in .NET, you should `Dispose` any `IDisposable` resources allocated on your behalf when you've finished using them. However, in Rx, it is common not to call `Dispose` on objects representing subscriptions, so if you implement such an object, you should not count on it being disposed. It's typically unnecessary, because Rx can clean up for you. Unlike with ordinary .NET events or delegates, observables can unambiguously come to an end, at which point any resources allocated to subscribers can be freed. (Some run indefinitely, but in that case, subscriptions usually remain active for the life of the program.) Admittedly, the examples I've shown so far don't clean up automatically, because I've provided my own implementations that are simple enough not to need to, but the Rx libraries do if you use their source and subscriber implementations. The only time you'd normally dispose of a subscription in Rx is if you want to unsubscribe before the source completes.

NOTE

Subscribers are not obliged to ensure that the `object` returned by `Subscribe` remains reachable. You can ignore it if you don't need the ability to unsubscribe early, and it won't matter if the garbage collector frees the object, because none of the `IDisposable` implementations that Rx supplies to represent subscriptions have finalizers. (And although you don't normally implement these yourself—I'm doing so here only to illustrate how it works—if you do write your own, you should take the same approach: do not implement a finalizer on a class that represents a subscription.)

The `KeyWatcher` class in [Example 11-7](#) has a `Run` method. That's not a standard Rx feature; it's just a loop that sits and waits for keyboard input—this observable won't actually produce any notifications unless something calls that method. Each time this loop receives a key, it calls the `OnNext` method on every currently subscribed observer. Notice that I'm building a copy of the subscriber list (by calling `ToArray`—that's a simple way to get a `List<T>` to duplicate its contents), because there's every possibility that a subscriber might choose to unsubscribe in the middle of a call to `OnNext`. If I had passed the subscriber list directly to `foreach`, I would get an exception in this scenario, because a `List<T>` doesn't allow items to be added and removed if you're in the middle of iterating through one.

WARNING

This example only guards against re-entrant calls on the same thread; handling multithreaded unsubscription would be altogether more complex. In fact, even building a copy is not sufficiently paranoid. I should really be checking that each observer in my snapshot is still currently subscribed before calling its `OnNext`, because it's possible that one observer might choose to unsubscribe some other observer. This also makes no attempt to deal with unsubscription from another thread. Later on, I'll replace all of this with a much more robust implementation from the Rx library.

In use, this hot source is very similar to my cold sources. We need to create an instance of the `KeyWatcher` and also another instance of my ob-

server class (with a type argument of `char` this time, because this source produces characters instead of strings). Because this source does not generate items until its monitoring loop runs, I need to call `Run` to kick it off, as [Example 11-8](#) does.

Example 11-8. Attaching an observer to an observable

```
var source = new KeyWatcher();  
var sub = new MySubscriber<char>();  
source.Subscribe(sub);  
source.Run();
```

Running that code, the application will wait for keyboard input, and if you press, say, the *m* key, the observer ([Example 11-2](#)) will display the message `Received: m`. (And since my source never ends, the `Run` method will never return.)

You might need to deal with a mixture of hot and cold observables. Also, some cold sources have some hot characteristics. For example, you could imagine a source that represented alert messages, and it might make sense to implement that in such a way that it stored alerts, to make sure you didn't miss anything that happens in between creating the source and attaching a subscriber. So it would be a cold source—any new subscriber would get all the events so far—but once a subscriber has caught up, the ongoing behavior would look more like a hot source, because any new events would be broadcast to all current subscribers. As you'll see, the Rx libraries provide various ways to mix and adapt between the two types of sources.

While it's useful to see what observers and observables need to do, it's more productive to let Rx take care of the grunt work, so now I'll show how you would write sources and subscribers if you were using the `System.Reactive` NuGet library instead of just the two fundamental interfaces.

Publishing and Subscribing with Delegates

If you use the `System.Reactive` NuGet package, you do not need to implement either `IObservable<T>` or `IObserver<T>` directly. The library provides several implementations. Some of these are adapters, bridging between Rx and other representations of asynchronously generated sequences. Some wrap existing observable streams. But the helpers aren't just for adapting existing things. They can also help if you want to write code that originates new items or that acts as the final destination for items. The simplest of these helpers provide delegate-based APIs for creating and consuming observable streams.

Creating an Observable Source with Delegates

Some of the preceding examples have shown that although `IObservable<T>` is a simple interface, sources that implement it may have to do a fair amount of work to track subscribers. And we've not even seen the whole story yet. As you'll see in [“Schedulers”](#), a source often needs to take extra measures to ensure that it integrates well with Rx's threading mechanisms. Fortunately, the Rx libraries can do some of that work for us. [Example 11-9](#) shows how to use the `Observable` class's static `Create` method to implement a cold source. (Each call to `GetFilePusher` will create a new source, so this is effectively a factory method.) `Observable` is defined in the `System.Reactive.Linq` namespace.

Example 11-9. Delegate-based observable source

```
public static IObservable<string> GetFilePusher(string path)
{
    return Observable.Create<string>(async (observer, cancel) =>
    {
        using (var sr = new StreamReader(path))
        {
            while (await sr.ReadLineAsync(cancel) is string line)
            {
```

```

        observer.OnNext(line);
    }
}
observer.OnCompleted();
});
}

```

This serves the same purpose as [Example 11-5](#)—it provides an observable source that supplies each line in a file in turn to subscribers. The heart of the code is the same, but I’ve been able to write just a single method instead of a whole class. Each time an observer subscribes to the observable that `GetFilePusher` returns, Rx invokes the callback I passed to `Create`, and that just has to provide the items. Rx supplies the `IObservable<T>` implementation, and also the `IDisposable` returned for each call to `Subscribe`.

I’ve also been able to use C#’s asynchronous language features (specifically, the `async` and `await` keywords, the subject of [Chapter 17](#)), which is helpful because this code does file I/O. If you don’t need this, `Create` offers overloads that work with non-`async` callbacks.

I’ve written rather less code than in [Example 11-5](#), but as well as simplifying my implementation, `Observable.Create` does three more subtle things for us that are not immediately apparent from the code.

First, this handles errors, even though this looks more like the non-error-aware [Example 11-5](#) than the more complex [Example 11-6](#). Rx will automatically handle exceptions that emerge from the callback and will report them to subscribers by calling `OnError`. The earlier example was complicated by the possibility of errors emerging from the subscriber’s `OnNext`, but now we don’t need to worry about that, because Rx does not pass the real `IObserver<T>` directly to our callback. The `observer` argument in the nested method in [Example 11-9](#) refers to an Rx-supplied wrapper that detects when the underlying `OnNext` throws an exception, and automatically shuts down the subscription before allowing the exception to propagate. This wrapper will ignore all further calls to any of the `IObserver<T>` methods, freeing us from the convoluted error handling we needed in [Example 11-6](#).

Second, this code handles unsubscription, unlike the earlier examples. The `Create` method passes a `CancellationToken` to notify us if the subscriber calls `Dispose` on the object returned by `Subscribe`. (Cancellation is described in [“Cancellation”](#).) [Example 11-9](#) passes that to `ReadLineAsync`, which means that work will stop immediately upon unsubscription. (`ReadLineAsync` will throw a `TaskCanceledException`, but Rx will handle that, and everything will come to a halt.) More subtly, Rx has our back even if we don’t pay full attention to the `CancellationToken`. The wrapper Rx supplies for the observer automatically stops forwarding notifications upon unsubscription. So even if the loop here hadn’t passed `cancel` to `ReadLineAsync`, and carried on running through the file even after the subscriber stopped listening, the subscriber wouldn’t receive items after it has asked to stop.

The third thing `Observable.Create` does for us under the covers is that in certain circumstances, it will use Rx’s scheduler system to call our code via a work queue instead of invoking it directly. This avoids deadlocks that could otherwise occur in cases where you’ve chained multiple observables together. I will be describing schedulers later in this chapter.

`Observable.Create` is good for cold sources such as [Example 11-9](#). Hot sources work differently, broadcasting live events to all subscribers, and `Observable.Create` does not cater to them directly because it invokes the delegate you pass once for each subscriber. However, the Rx libraries can still help.

Rx provides a `Publish` extension method for any `IObservable<T>`, defined by the `Observable` class in the `System.Reactive.Linq` namespace. This method is designed to wrap a source whose subscription method (i.e., the delegate you pass to `Observable.Create`) supports being run only once but to which you want to attach multiple subscribers—it handles the multicast logic for you. Strictly speaking, a source that supports only a single subscription has not fulfilled the `IObservable<T>` interface’s contract, but as long as you hide it behind `Publish`, it doesn’t matter. The purpose of `Publish` is to let you write a source that falls short in this particular way, and to turn it into a proper a hot source.

[Example 11-10](#) shows how to create a source that provides the same func-

tionality as the `KeyWatcher` in [Example 11-7](#). I've also hooked up two subscribers, just to illustrate the point that this supports multiple subscribers.

Example 11-10. Delegate-based hot source

```
IObservable<char> singularHotSource = Observable.Create(
    (Func<IObserver<char>, IDisposable>) (observer =>
    {
        while (true)
        {
            observer.OnNext(Console.ReadKey(true).KeyChar);
        }
    }));

IConnectableObservable<char> keySource = singularHotSource.Publish();

keySource.Subscribe(new MySubscriber<char>());
keySource.Subscribe(new MySubscriber<char>());

keySource.Connect();
```

The `Publish` method does not call `Subscribe` on the source immediately. Nor does it do so when you first attach a subscriber to the source it returns. I have to tell the published source when I want it to start. Notice that `Publish` returns an `IConnectableObservable<T>`. This derives from `IObservable<T>` and adds a single extra method, `Connect`. This interface represents a source that doesn't start until it's told to, and it's designed to let you hook up all the subscribers you need before you set it running. Calling `Connect` on the source returned by `Publish` causes it to subscribe to my original source, invoking the subscription callback I passed to `Observable.Create`, which runs my loop. This causes the `Connect` method to have the same effect as calling `Run` on my original [Example 11-7](#).

`Connect` returns an `IDisposable`. This provides a way to disconnect at some later point—that is, to unsubscribe from the underlying source. (If you don't call this, the connectable observable returned by `Publish` will remain subscribed to your source even if you `Dispose` each of the indi-

vidual downstream subscriptions.) In this particular example, the call to `Connect` will never return, because the code I passed to `Observable.Create` also never returns. Most observable sources don't do this. Typically, they avoid it by using either asynchronous or scheduler-based techniques, which I will show later in this chapter.

The combination of the delegate-based `Observable.Create` and the multicasting offered by `Publish` has enabled me to throw away everything in [Example 11-7](#) except for the loop that actually generates items, and even that has become simpler. Being able to remove about 80% of the code isn't the whole story, either. This will work better—`Publish` lets Rx handle my subscribers, which will deal correctly with the awkward situations in which subscribers unsubscribe while being notified.

Of course, the Rx libraries don't just help with implementing sources. They can simplify subscribers too.

Subscribing to an Observable Source with Delegates

Just as you don't have to implement `IObservable<T>`, it's also not necessary to provide an implementation of `IObserver<T>`. You won't always care about all three methods—the `KeyWatcher` observable in [Example 11-7](#) never even calls the `OnCompleted` or `OnError` methods, because it runs indefinitely and has no error detection. Even when you do need to provide all three methods, you won't necessarily want to write a whole separate type to provide them. So the Rx libraries provide extension methods to simplify subscription, defined by the `ObservableExtensions` class in the `System` namespace. Most C# source files include a `using System;` directive, or are in a project with an implicit global `using` directive for `System`, so the extensions it offers will usually be available as long as your project has a reference to the `System.Reactive` NuGet package. There are several overloads for the `Subscribe` method available for any `IObservable<T>`. [Example 11-11](#) uses one of them.

Example 11-11. Subscribing without implementing `IObserver<T>`

```
var source = new KeyWatcher();  
source.Subscribe(value => Console.WriteLine("Received: " + value));  
source.Run();
```

This example has the same effect as [Example 11-8](#). However, by using this approach, we no longer need to write a whole class implementing `IObserver<T>` like [Example 11-2](#). With this `Subscribe` extension method, Rx provides the `IObserver<T>` implementation for us, and we provide methods only for the notifications we want.

The `Subscribe` overload used by [Example 11-11](#) takes an `Action<T>`, where `T` is the item type of the `IObservable<T>`, which in this case is `char`. Although my source doesn't provide error notifications or use `OnCompleted` to indicate the end of the items, plenty of sources do, so there are three overloads of `Subscribe` to handle that. One takes an extra delegate of type `Action<Exception>` to handle errors. Another takes a second delegate of type `Action` (i.e., one that takes no arguments) to handle the completion notification. The third overload takes three delegates—the same per-item callback that they all take, and then an exception handler and a completion handler.

NOTE

If you do not provide an exception handler when using delegate-based subscription, but the source calls `OnError`, the `IObserver<T>` Rx supplies throws the exception to keep the error from going unnoticed. [Example 11-5](#) calls `OnError` in the `catch` block where it handles I/O exceptions, and if you subscribed using the technique in [Example 11-11](#), you'd find that the call to `OnError` throws the `IOException` right back out again—the same exception is thrown twice in a row, once by the `StreamReader` and then again by the Rx-supplied `IObserver<T>` implementation. Since we'd already be in the `catch` block in [Example 11-5](#) by this time (and not the `try` block), this second throw would cause the exception to emerge from the `Subscribe` method, either to be handled farther up the stack or crashing the application.

There's one more overload of the `Subscribe` extension method, taking no arguments. This subscribes to a source and then does nothing with the items it receives. (It will throw any errors back to the source, just like the other overloads that don't take an error callback.) This would be useful if you have a source that does something important as a side effect of subscription, although it's probably best to avoid designs where that's necessary.

Sequence Builders

Rx defines several methods that create new sequences from scratch, without requiring either custom types or callbacks. These are designed for certain simple scenarios such as single-element sequences, empty sequences, or particular patterns. These are all static methods defined by the `Observable` class.

Empty

The `Observable.Empty<T>` method is similar to the `Enumerable.Empty<T>` method from LINQ to Objects that I showed in [Chapter 10](#): it produces an empty sequence. (The difference, of course, is that it implements `IObservable<T>`, not `IEnumerable<T>`.) As with the LINQ to Objects method, this is useful when you're working with APIs that demand an observable source and you have no items to provide.

Any observer that subscribes to an `Observable.Empty<T>` sequence will have its `OnCompleted` method called immediately.

Never

The `Observable.Never<T>` method produces a sequence that never does anything—it produces no items, and unlike an empty sequence, it never even completes. (The Rx team considered calling this `Infinite<T>` to emphasize the fact that as well as never producing anything, it also never ends.) There is no counterpart in LINQ to Objects. If you wanted to write an `IEnumerable<T>` equivalent of `Never`, it would be one that blocked

indefinitely when you first tried to retrieve an item. In the pull-based world of LINQ to Objects, this would not be at all useful—it would cause the calling thread to freeze for the lifetime of the process. (An `IAsyncEnumerable<T>` equivalent would return a `ValueTask<bool>` that never completes from the first call to `MoveNextAsync`. This does not need to block a thread, but you still end up with a logical operation in progress that never completes.) But in Rx's reactive world, sources don't block progress just because they are in a state where they're not currently producing items, so `Never` is a less disastrous idea. It can be helpful with some of the operators I'll show later that can use an `IObservable<T>` to represent duration. `Never` can represent an activity you want to run indefinitely.

Return

The `Observable.Return<T>` method takes a single argument and returns an observable sequence that immediately produces that one value and then completes. Just as `Empty` is useful when something requires a sequence and you have no items, this is useful when something requires a sequence and you have exactly one item. This is a cold source—you can subscribe to it any number of times, and each subscriber will receive the same value. There is no exact equivalent in LINQ to Objects, although the Rx team provides a library called the Interactive Extensions for .NET (or `Ix` for short, available in the `System.Interactive` NuGet package) that provides `IEnumerable<T>` versions of this and several of the other operators described in this chapter that are in Rx but not LINQ to Objects.

Throw

The `Observable.Throw<T>` method takes a single argument of type `Exception` and returns an observable sequence that passes that exception to `OnError` immediately for any subscriber. Like `Return`, this is also a cold source that can be subscribed to any number of times, and it will do the same thing to each subscriber.

Range

The `Observable.Range` method generates a sequence of numbers. (It always returns an `IObservable<int>`, which is why it does not take a type argument.) Like the `Enumerable.Range` method, it takes a starting number and a count. This is a cold source that will produce the entire range for each subscriber.

Repeat

The `Observable.Repeat<T>` method takes an input and produces a sequence that repeatedly produces that input over and over again. The input can be a single value, but it can also be another observable sequence, in which case it will forward items until that input completes and will then resubscribe to produce the whole sequence repeatedly. (That means that this will only genuinely repeat the data if you pass it a cold observable.)

If you pass no other arguments, the resulting sequence will produce values indefinitely—the only way to stop it is to unsubscribe. You can also pass a count, saying how many times you would like the input to repeat.

Generate

The `Observable.Generate<TState, TResult>` method can produce more complex sequences than the other methods I've just described. You provide `Generate` with an object or value representing the generator's initial state. This can be any type you like—it's one of the method's generic type arguments. You must also supply three functions: one that inspects the current state to decide whether the sequence is complete yet, one that advances the state in preparation for producing the next item, and one that determines the value to produce for the current state. [Example 11-12](#) uses this to create a source that produces random numbers until the sum total of all the numbers produced exceeds 10,000.

Example 11-12. Generating items

```
IObservable<int> src = Observable.Generate(  
    (Current: 0, Total: 0, Random: new Random()),  
    state => state.Total <= 10000,  
    state =>  
    {  
        int value = state.Random.Next(1000);  
        return (value, state.Total + value, state.Random);  
    },  
    state => state.Current);
```

This always produces 0 as the first item, illustrating that `Generate` calls the function that determines the current value (the final lambda in [Example 11-12](#)) before making the first call to the function that iterates the state.

You could achieve the same effect as this example by using `Observable.Create` and a loop. However, `Generate` inverts the flow of control: instead of your code sitting in a loop telling Rx when to produce the next item, Rx asks your functions for the next item. This gives Rx more flexibility over scheduling of the work. For example, it enables `Generate` to offer overloads that bring timing into the picture.

[Example 11-13](#) produces items in a similar way but passes an extra function as the final argument that tells Rx to delay the delivery of each item by a random amount.

Example 11-13. Generating timed items

```
IObservable<int> src = Observable.Generate(  
    (Current: 0, Total: 0, Random: new Random()),  
    state => state.Total < 10000,  
    state =>  
    {  
        int value = state.Random.Next(1000);  
        return (value, state.Total + value, state.Random);  
    },  
    state => state.Current,  
    state => TimeSpan.FromMilliseconds(state.Random.Next(1000)));
```

For this to work, Rx needs to be able to schedule work to happen at some point in the future. I'll explain how this works in [“Schedulers”](#).

LINQ Queries

One of the greatest benefits of using Rx is that it has a LINQ implementation, enabling you to write queries to process asynchronous streams of items such as events. [Example 11-14](#) illustrates this. It begins by producing an observable source representing `MouseMove` events from a UI element. I'll talk about this technique in more detail in [“Adaptation”](#), but for now it's enough to know that Rx can wrap any .NET event as an observable source. Each event produces an item that provides two properties containing the values normally passed to event handlers as arguments (i.e., the sender and the event arguments).

Example 11-14. Filtering items with a LINQ query

```
IObservable<EventPattern<MouseEventArgs>> mouseMoves =  
    Observable.FromEventPattern<MouseEventHandler, MouseEventArgs>(  
        h => background.MouseMove += h, h => background.MouseMove -= h);  
  
IObservable<Point> dragPositions =  
    from move in mouseMoves  
    where Mouse.Captured == background  
    select move.EventArgs.GetPosition(background);  
  
dragPositions.Subscribe(point => { line.Points.Add(point); });
```

The `where` clause in the LINQ query filters the events so that we process only those events that were raised while a specific UI element (`background`) has captured the mouse. This particular example is based on WPF (and that `background` variable comes from a XAML file that you can find in the full examples for this book), but in general, Windows desktop applications that want to support dragging *capture* the mouse when the mouse button is pressed and *release* it afterward. This ensures that the capturing element receives mouse move events for as long as the drag

is in progress, even if the mouse moves over other UI elements. Typically, UI elements receive mouse move events when the mouse is over them even if they have not captured the mouse. So I need that `where` clause in [Example 11-14](#) to ignore those events, leaving only mouse movements that occur while a drag is in progress. So, for the code in [Example 11-14](#) to work, you'd need to attach event handlers such as those in [Example 11-15](#) to the relevant element's `MouseDown` and `MouseUp` events.

Example 11-15. Capturing the mouse

```
private void OnBackgroundMouseDown(object sender, MouseButtonEventArgs e)
{
    background.CaptureMouse();
}

private void OnBackgroundMouseUp(object sender, MouseButtonEventArgs e)
{
    if (Mouse.Captured == background)
    {
        background.ReleaseMouseCapture();
    }
}
```

The `select` clause in [Example 11-14](#) works in Rx just like it does in LINQ to Objects, or with any other LINQ provider. It allows us to extract information from the source items to use as the output. In this case, `mouseMoves` is an observable sequence of `EventArgs` objects, but what I really want is an observable sequence of mouse locations. So the `select` clause in [Example 11-14](#) asks for the position relative to a particular UI element.

The upshot of this query is that `dragPositions` refers to an observable sequence of `Point` values, which will report each change of mouse position that occurs while a particular UI element in my application has captured the mouse. This is a hot source, because it represents something that's happening live: mouse input. The LINQ filtering and projection operators do not change the nature of the source, so if you apply them to a

hot source, the resulting query will also be hot, and if the source is cold, the filtered result will be too.

WARNING

Operators do not detect the hotness of the source. The `Where` and `Select` operators just pass this aspect straight through. Each time you subscribe to the final query produced by the `Select` operator, it will subscribe to its input. In this case, the input was the observable returned by the `Where` operator, which will in turn subscribe to the source produced by adapting the mouse move events. If you subscribe a second time, you'll get a second chain of subscriptions. The hot event source will broadcast every event to both chains, so each item will go through the filtering and projection process twice. So be aware that attaching multiple subscribers to a complex query of a hot source will work but may incur unnecessary expense. If you need to do this, it may be better to call `Publish` on the query, which as you've seen, can make a single subscription to its input and then multicast each item to all its subscribers.

The final line of [Example 11-14](#) subscribes to the filtered and projected source and adds each `Point` value it produces to the `Points` collection of another UI element called `line`. That's a `Polyline` element, not shown here,¹ and the upshot of this is that you can scrawl on the application's window with the mouse. (If you've been doing Windows development for long enough, you may remember the Scribble examples—the effect here is much the same.)

Rx provides most of the standard query operators described in [Chapter 10](#).² Most of these work in Rx exactly as they do with other LINQ implementations. However, some work in ways that may seem slightly surprising at first glance, as I will describe in the next few sections.

Grouping Operators

The standard grouping operator, `GroupBy`, produces a sequence of sequences. With LINQ to Objects, it returns `IEnumerable<IGrouping<TKey, TSource>>`, and as you saw in [Chapter 10](#), `IGrouping<TKey, TSource>` itself derives from `IEnumerable<TSource>`. The `GroupJoin` is similar in

concept: although it returns a plain `IEnumerable<T>`, that `T` is the result of a projection function that is passed a sequence as input. So, in either case, you get what is logically a sequence of sequences.

In the world of Rx, grouping produces an observable sequence of observable sequences. This is perfectly consistent but can seem a little surprising because Rx introduces a temporal aspect: the observable source that represents all the groups produces a new item (a new observable source) at the instant it discovers each new group. [Example 11-16](#) illustrates this by watching for changes in the filesystem and then forming them into groups based on the folder in which each occurred. For each group, we get an `IGroupedObservable<TKey, TSource>`, which is the Rx equivalent of `IGrouping<TKey, TSource>`.

Example 11-16. Grouping events

```
string path = Environment.GetFolderPath(Environment.SpecialFolder.MyDocuments);
var w = new FileSystemWatcher(path);
IObservable<EventPattern<FileSystemEventArgs>> changes =
    Observable.FromEventPattern<FileSystemEventHandler, FileSystemEventArgs>(
        h => w.Changed += h, h => w.Changed -= h);
w.IncludeSubdirectories = true;
w.EnableRaisingEvents = true;

IObservable<IGroupedObservable<string, string>> folders =
    from change in changes
    group Path.GetFileName(change.EventArgs.FullPath)
    by Path.GetDirectoryName(change.EventArgs.FullPath);

folders.Subscribe(f =>
{
    Console.WriteLine("New folder ({0})", f.Key);
    f.Subscribe(file =>
        Console.WriteLine("File changed in folder {0}, {1}", f.Key, file));
});
```

The lambda that subscribes to the grouping source, `folders`, subscribes to each group that the source produces. The number of folders from which events could occur is endless, as new ones could be added while

the program is running. So the `folders` observable will produce a new observable source each time it detects a change in a folder it hasn't seen before, as [Figure 11-2](#) shows.

Notice that the production of a new group doesn't mean that any previous groups are now complete, which is different than how we typically consume groups in LINQ to Objects. When you run a grouping query on an `IEnumerable<T>`, as it produces each group you can enumerate the contents entirely before moving on to the next one. But you can't do that with Rx, because each group is represented as an observable, and observables aren't finished until they tell you they're complete—instead, each group subscription remains active. In [Example 11-16](#), it's entirely possible that a folder for which a group had already started will be dormant for a long time while activity occurs in other folders, only for it to start up again later. And more generally, Rx's grouping operators have to be prepared for that to happen with any source.

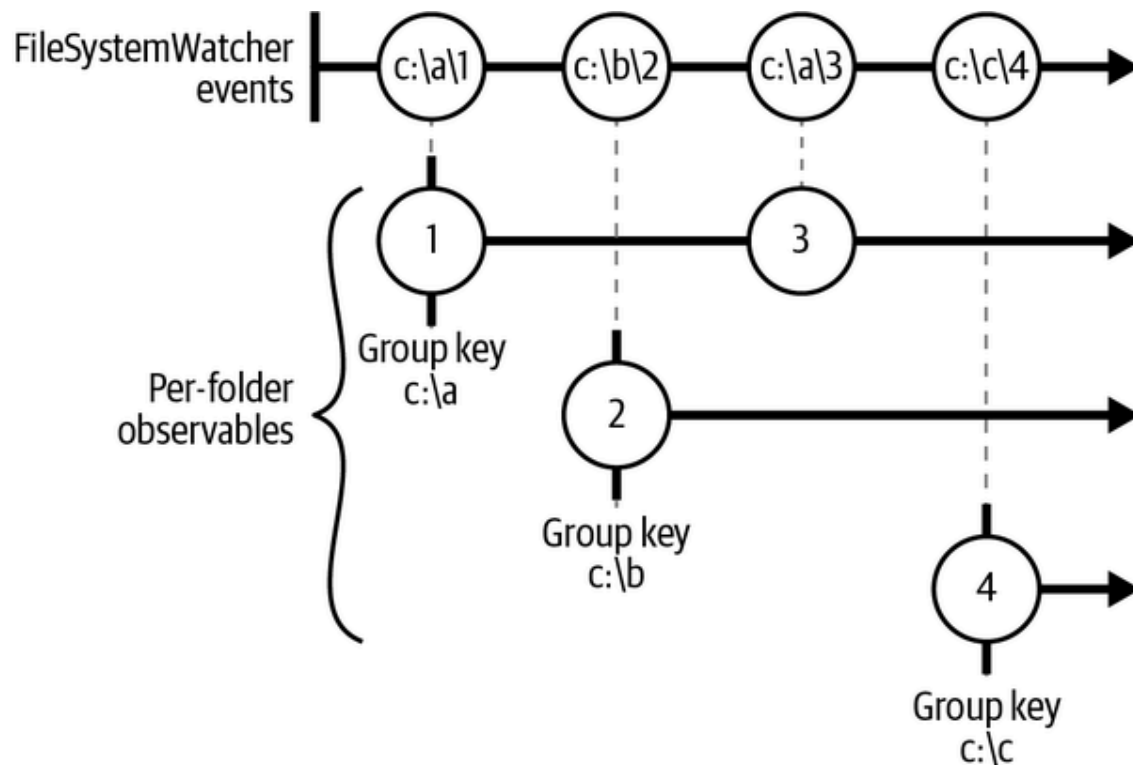


Figure 11-2. Splitting an `IObservable<T>` into groups

Join Operators

Rx provides the standard `Join` and `GroupJoin` operators. However, they work a bit differently than how LINQ to Objects or most database LINQ

providers handle joins. In those worlds, items from two input sets are typically joined based on having some value in common. However, Rx does not base joins on values. Instead, items are joined if they are contemporaneous—if their durations overlap, then they are joined.

But hang on a minute. What exactly is an item's duration? Rx deals in instantaneous events; producing an item, reporting an error, and finishing a stream are all things that happen at a particular moment. So the join operators use a convention: for each source item, you can provide a function that returns an `IObservable<T>`. The duration for that source item starts when the item is produced and finishes when the corresponding `IObservable<T>` first reacts (i.e., it either completes or generates an item). [Figure 11-3](#) illustrates this idea. At the top is an observable source, beneath which is a series of sources that define each item's duration. At the bottom, I've shown the duration that the per-item observables establish for their source items.

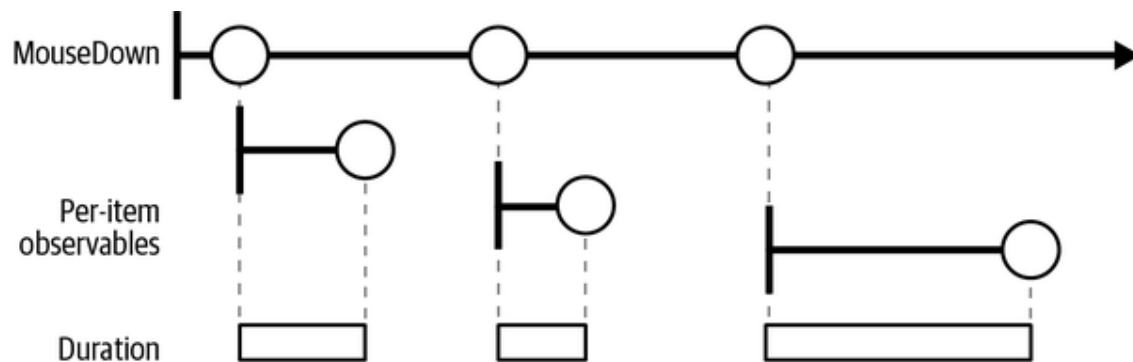


Figure 11-3. Defining duration with an `IObservable<T>` for each source item

Although you can use a different `IObservable<T>` for each source item, you don't have to—it's valid to use the same source every time. We could use a single source representing `MouseDown` events for all of the duration-defining observables in [Figure 11-3](#). A source can even define its own duration. For example, if you provide an observable source representing `MouseDown` events, you might want each item's duration to end when the next item begins. This would mean that the items had contiguous durations—after the first item arrives, there is always exactly one current item, and it is the last one that occurred.

Now that we know how Rx decides what constitutes an item's duration for the purposes of a join, how does it use that information? Remember, join operators combine two inputs. (The duration-defining sources do not count as an input. They provide additional information about one of the inputs.) Rx considers a pair of items from the two input streams to be related if their durations overlap. The way it presents related items in the output depends on whether you use the `Join` or the `GroupJoin` operator. The `Join` operator's output is a stream containing one item for each pair of related items. (You provide a projection function that will be passed each pair, and it's up to you what to do with them. This function gets to decide the output item type for the joined stream.) [Figure 11-4](#) shows two input streams based on the events `MouseDown` and `MouseMove` (with durations defined by `MouseUp` and `MouseMove`, respectively). At the bottom of the diagram is the observable the `Join` operator would produce for these two streams.

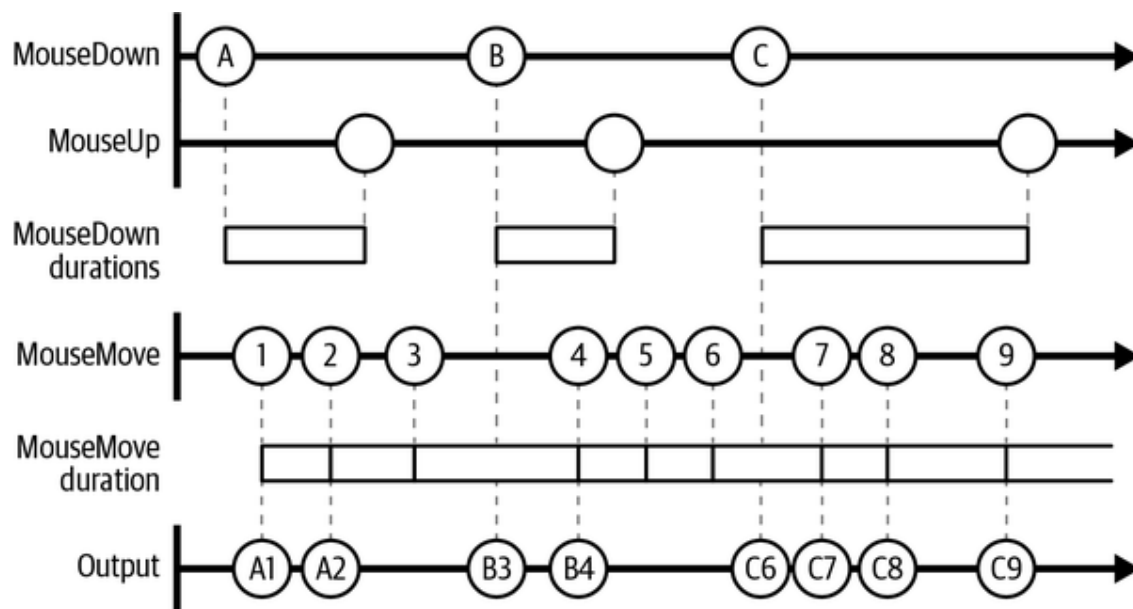


Figure 11-4. `Join` operator

As you can see, any place where the durations of two items from the input streams overlap, we get an output item combining the two inputs. If the overlapping items started at different times (which will normally be the case), the output item is produced whenever the later of the two inputs started. The `MouseDown` event A starts before the `MouseMove` event 1, so the resulting output, A1, occurs where the overlap begins (i.e., when `MouseMove` event 1 occurs). But event 3 occurs before event B, so the joined output B3 occurs when B starts.

Event 5's duration does not overlap with any `MouseDown` items' durations, so we do not see any items for that in the output stream.

Conversely, it would be possible for a `MouseMove` event to appear in multiple output items (just like each `MouseDown` event does). If there had been no 3 event, event 2 would have a duration that started inside A and finished inside B, so as well as the A2 shown in [Figure 11-4](#), there would be a B2 event at the same time as B starts. [Example 11-17](#) shows code that performs the join illustrated in [Figure 11-4](#).

Example 11-17. Joining observables

```
IObservable<EventPattern<MouseEventArgs>> downs =  
    Observable.FromEventPattern<MouseButtonEventHandler, MouseEventArgs>(  
        h => background.MouseDown += h, h => background.MouseDown -= h);  
IObservable<EventPattern<MouseEventArgs>> ups =  
    Observable.FromEventPattern<MouseButtonEventHandler, MouseEventArgs>(  
        h => background.MouseUp += h, h => background.MouseUp -= h);  
IObservable<EventPattern<MouseEventArgs>> allMoves =  
    Observable.FromEventPattern<MouseEventHandler, MouseEventArgs>(  
        h => background.MouseMove += h, h => background.MouseMove -= h);  
  
IObservable<Point> dragPositions = downs.Join(  
    allMoves,  
    down => ups,  
    move => allMoves,  
    (down, move) => move.EventArgs.GetPosition(background));
```

We can use the `dragPositions` observable source produced by either of these examples to replace the one in [Example 11-14](#). Unlike some earlier examples that needed to filter based on whether the `background` element has captured the mouse, Rx is now providing us only those move events whose duration overlaps with the duration of a `MouseDown` event. Any moves that happen in between mouse presses will either be ignored or, if they are the last move to occur before a mouse down, we'll receive that position at the moment the mouse button is pressed. The effect is that `dragPositions` reports all the mouse locations from the start to end of any drag operation.

`GroupJoin` combines items in a similar way, but instead of producing a single observable output, it produces an observable of observables. For the current example, that would mean that its output would produce a new observable source for each `MouseDown` input. This would consist of all the pairs containing that input, and it would have the same duration as that input.

SelectMany Operator

As you saw in [Chapter 10](#), the `SelectMany` operator effectively flattens a collection of collections into a single one. This operator gets used when a query expression has multiple `from` clauses, and with LINQ to Objects, its operation is similar to having nested `foreach` loops. With Rx, it still has this flattening effect—it lets you take an observable source where each item it produces is also an observable source (or can be used to generate one), and the result of the `SelectMany` operator will be a single observable sequence that contains all of the items from all of the child sources. However, as with grouping, things may be less orderly than in LINQ to Objects. The push-driven nature of Rx, with its potential for asynchronous operation, makes it possible for all of the observable sources involved to be pushing new items at once, including the original source that is used as a source of nested sources. (The `SelectMany` operator still ensures that only one event will be delivered at a time—when it calls on `OnNext`, it waits for that to return before making another call. The potential for chaos only goes as far as mixing up the order in which events are delivered.)

When you use LINQ to Objects to iterate through a jagged array, everything happens in a straightforward order. It will retrieve the first nested array and then iterate through all the elements in that array before moving to the next nested array and iterating through that, and so on. But this orderly flattening only occurs because with `IEnumerable<T>`, the consumer controls when it retrieves items. With Rx, subscribers receive items when sources provide them.

Despite the free-for-all, the behavior is straightforward enough: the output stream produced by `SelectMany` just provides items as and when the

sources provide them.

Aggregation and Other Single-Value Operators

Several of the standard LINQ operators reduce an entire sequence of values to a single value. These include the aggregation operators, such as `Min`, `Sum`, and `Aggregate`; the quantifiers `Any` and `All`; and the `Count` operator. It also includes selective operators, such as `ElementAt`. These are available in Rx, but unlike most LINQ implementations, the Rx implementations do not return plain single values. They all return an `IObservable<T>`, just like operators that produce sequences as outputs.

NOTE

The `First`, `Last`, `FirstOrDefault`, `LastOrDefault`, `Single`, and `SingleOrDefault` operators should all work the same way, but for historical reasons, they do not. Introduced in v1 of Rx, they returned single values that were not wrapped in an `IObservable<T>`, which meant they would block until the source provided what they needed. This doesn't fit well with a push-based model and risks introducing deadlock, so these are now deprecated, and there are new asynchronous versions that work the same way as the other single-value operators in Rx. These all just append `Async` to the original operators' names (e.g., `FirstAsync`, `LastAsync`, etc.).

Each of these operators still produces a single value, but they all present that value as an observable source. The reason is that unlike LINQ to Objects, Rx cannot enumerate its input to calculate the aggregate value or to find the value being selected. The source is in control, so the Rx versions of these operators have to wait for the source to provide its values—like all operators, the single-value operators have to be reactive, not proactive. Operators that need to see every value, such as `Average`, cannot produce their result until the source says it has finished. Even an operator that doesn't need to wait until the very end of the input, such as `FirstAsync` or `ElementAt`, still cannot do anything until the source decides to provide the value the operator is waiting for. As soon as a single-value operator is able to provide a value, it does so and then completes.

The `ToArray`, `ToList`, `ToDictionary`, and `ToLookup` operators work in a similar way. Although these all produce the entire contents of the source, they do so as a single output object, which is wrapped as a single-item observable source.

If you really want to sit and wait for the value of any of these items, you can use C#'s `await` keyword on any observable source. Logically, it does the same kind of thing as the old deprecated `First`, `Last`, etc. methods but it does so with an efficient nonblocking asynchronous wait of the kind described in [Chapter 17](#). (If you use it on a source that returns multiple items, it waits until the source completes and then returns the final item. It throws an exception if the source completes without producing any items.) And if you are truly determined to risk deadlock by blocking your thread while waiting for value, you can use `Wait`, a nonstandard operator specific to Rx available on any `IObservable<T>`. So the nonasynchronous “sit and wait” behavior of the deprecated `First`, `Last`, etc., operators is still available; it's just no longer the default.

Concat Operator

Rx's `Concat` operator shares the same concept as other LINQ implementations: it combines two input sequences to produce a sequence that will produce every item in its first input, followed by every item in its second input. (In fact, Rx goes further than some LINQ providers and can accept a collection of inputs and will concatenate them all.) This is useful only if the first stream eventually completes—that's true in LINQ to Objects too, of course, but infinite sources are more common in Rx. Also, be aware that this operator does not subscribe to the second stream until the first has finished. This is because cold streams typically start producing items when you subscribe, and the `Concat` operator does not want to have to buffer the second source's items while it waits for the first to complete. This means that `Concat` may produce nondeterministic results when used with hot sources. (If you want an observable source that contains all the items from two hot sources, use `Merge`, which I'll describe shortly.)

Rx is not satisfied with merely providing standard LINQ operators. It defines many more of its own operators.

Rx Query Operators

One of Rx's main goals is to simplify working with multiple potentially independent observable sources that produce items asynchronously. Rx's designers sometimes refer to "orchestration and synchronization," meaning that your system may have many things going on at once, but you need to achieve some kind of coherency in how your application reacts to events. Many of Rx's operators are designed with this goal in mind.

NOTE

Not everything in this section is driven by the unique requirements of Rx. A few of Rx's nonstandard operators (e.g., `Scan`) would make perfect sense in other LINQ providers. And versions of many of these are available for `IEnumerable<T>` in the Interactive Extensions for .NET (Ix), which, as mentioned earlier, are to be found in the `System.Interactive` NuGet package.

Rx has such a large repertoire of operators that to do them all justice would roughly quadruple the size of this chapter, which is already on the long side. Since this is not a book about Rx, and because some of the operators are very specialized, I will just pick some of the most useful. I recommend browsing through [the source](#) or the [Rx documentation](#) to discover the full and remarkably comprehensive set of operators it provides.

Merge

The `Merge` operator combines all of the elements from two or more observable sequences into a single observable sequence. I can use this to fix a problem that occurs in [Example 11-14](#). This processes mouse input, and if you've done much Windows UI programming, you know that you will not necessarily get a mouse move notification corresponding to the points at which the mouse button was pressed and released. The notifications for these button events include mouse location information, so Windows sees no need to send a separate mouse move message providing these locations, because it would just be sending you the same information twice. This is perfectly logical, and also rather annoying.³ These start and end lo-

cations are not in the observable source that represents mouse positions in those examples. I can fix that by merging the positions from all three events. [Example 11-18](#) shows how to fix [Example 11-14](#).

Example 11-18. Merging observables

```
IObservable<EventPattern<MouseEventArgs>> dragMoves =  
    from move in allMoves  
    where Mouse.Captured == background  
    select move;  
  
IObservable<EventPattern<MouseEventArgs>> allDragPositionEvents =  
    Observable.Merge(downs, ups, dragMoves);  
  
IObservable<Point> dragPositions =  
    from move in allDragPositionEvents  
    select move.EventArgs.GetPosition(background);
```

This uses the three observables from earlier examples representing the three relevant events: `MouseDown`, `MouseUp`, and `MouseMove`. Since all three of these need to share the same projection (the `select` clause), but only one needs to filter events, I've restructured things a bit. Only mouse moves need filtering, so I've written a separate query for that. I've then used the `Observable.Merge` method to combine all three event streams into one.

NOTE

`Merge` is available both as an extension method and a nonextension `static` method. If you use the extension methods available on a single observable, the only `Merge` overloads available combine it with a single other source (optionally specifying a scheduler). In this case, I had three sources, which is why I used the nonextension method form. However, if you have an expression that is either an enumerable of observable sources or an observable source of observable sources, you'll find that there are also `Merge` extension methods for these. So I could have written `new[] { downs, ups, dragMoves }.Merge()`.

My `allDragPositionEvents` variable refers to a single observable stream that will report all the mouse moves I need. Finally, I run this through a projection to extract the mouse position for each item. Again, the result is a hot source. As before, it will produce a position any time the mouse moves while the `background` element has captured the mouse, but it will also produce a position each time either the `MouseDown` or `MouseUp` event occurs. I could subscribe to this with the same call shown in the final line of [Example 11-14](#) to keep my UI up to date, and this time, I wouldn't be missing the start and end positions.

In the example I've just shown, the sources are all endless, but that will not always be the case. What should a merged observable do when one of its inputs stops? If one stops due to an error, that error will be passed on by the merged observable, at which point it will be complete—an observable is not allowed to continue producing items after reporting an error. However, although an input can unilaterally terminate the output with an error, if inputs complete normally, the merged observable doesn't complete until all of its inputs are complete.

Windowing Operators

Rx defines two operators, `Buffer` and `Window`, that both produce an observable output where each item is based on multiple adjacent items from the source. (The name `Window` has nothing to do with UIs, by the way.) [Figure 11-5](#) shows three ways in which you could use the `Buffer` operator. I've numbered the circles representing items in the input, and below this are blobs representing the items that will emerge from the observable source produced by `Buffer`, with lines and numbers indicating which input items are associated with each output item. `Window` works in a very similar way, as you'll see shortly.

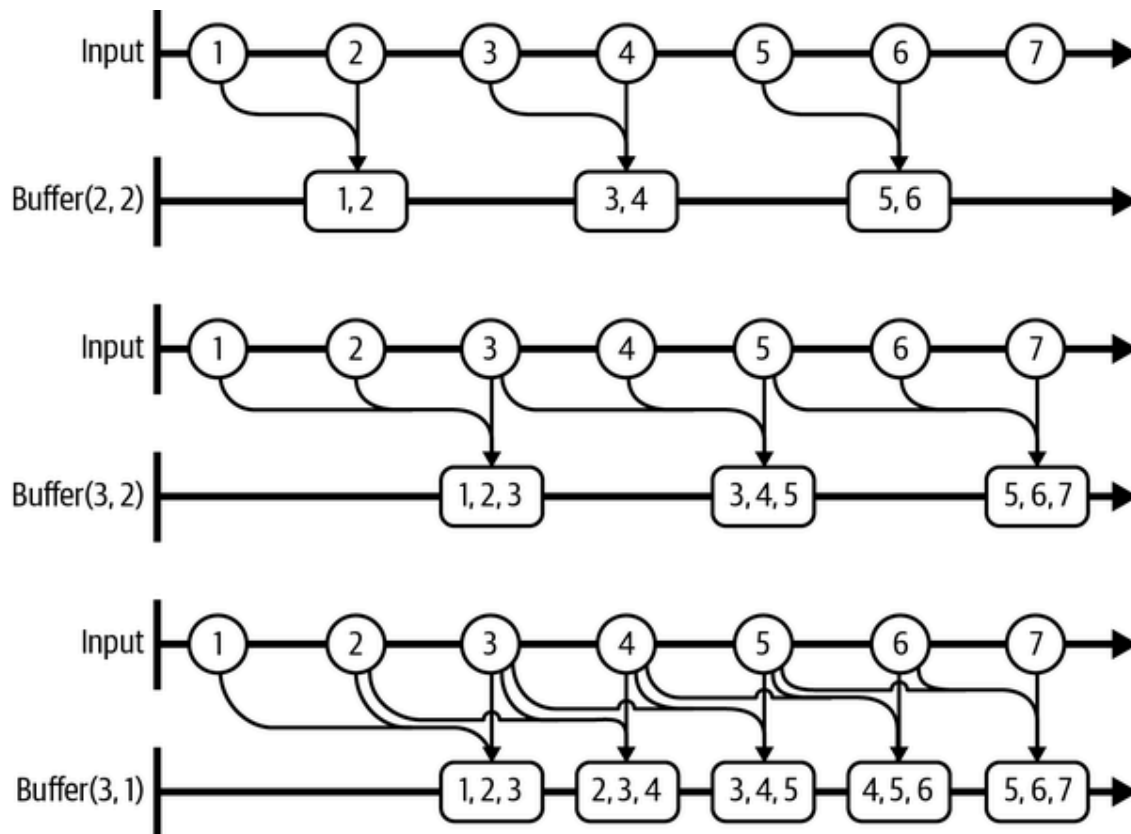


Figure 11-5. Sliding windows with the `Buffer` operator

In the first case, I've passed arguments of `(2, 2)`, indicating that I want each output item to correspond to two input items and that I want to start a new buffer on every second input item. That may sound like two different ways of saying the same thing until you look at the second example in [Figure 11-5](#), in which arguments of `(3, 2)` indicate that each output item corresponds to three items from the input, but I still want the buffers to begin on every other input. This means that each *window*—the set of items from the input used to build an output item—overlaps with its neighbors. This will happen whenever the second argument, the *skip*, is smaller than the window. The first output item's window contains the first, second, and third input. The second output's window contains the third, fourth, and fifth, so the third item appears in both.

The final example in the figure shows a window size of three, but this time I've asked for a skip size of one—so in this case, the window moves along by only one input item at a time, but it incorporates three items from the source each time. I could also specify a skip that is larger than the window, in which case the input items that fell between windows would simply be ignored.

The `Buffer` operator tends to introduce a lag. In the second and third cases, the window size of three means that the input observable needs to produce its third value before the whole window can be provided for the output item. With `Buffer`, this always means a delay of the size of the window, but as you'll see, with the `Window` operator, each window can get under way before it is full.

NOTE

`Buffer` offers an overload that takes a single number, which has the same effect as passing the same number twice. (E.g., instead of `Buffer(2, 2)`, you could write just `Buffer(2)`.) This is logically equivalent to LINQ to Objects' `Chunk` operator. As discussed in [Chapter 10](#), the main reason Rx didn't use the same name is that Rx implemented `Buffer` about a decade before LINQ to Objects added `Chunk`.

The difference between the `Buffer` and `Window` operators is the way in which they present the windowed items. `Buffer` is the most straightforward. It provides an `IObservable<IList<T>>`, where `T` is the input item type. In other words, if you subscribe to the output of `Buffer`, for each window produced, your subscriber will be passed a list containing all the items in the window. [Example 11-19](#) uses this to produce a smoothed-out version of the mouse locations from [Example 11-14](#).

Example 11-19. Smoothing input with `Buffer`

```
IObservable<Point> smoothed = from points in dragPositions.Buffer(5, 2)
                              let x = points.Average(p => p.X)
                              let y = points.Average(p => p.Y)
                              select new Point(x, y);
```

The first line of this query states that I want to see groups of five consecutive mouse locations, and I want one group for every other input. The rest of the query calculates the average mouse position within the window and produces that as the output item. [Figure 11-6](#) shows the effect. The top line is the result of using the raw mouse positions. The line immediately beneath it uses the smoothed points generated by [Example 11-19](#)

from the same input. As you can see, the top line is rather ragged, but the bottom line has smoothed out a lot of the lumps.

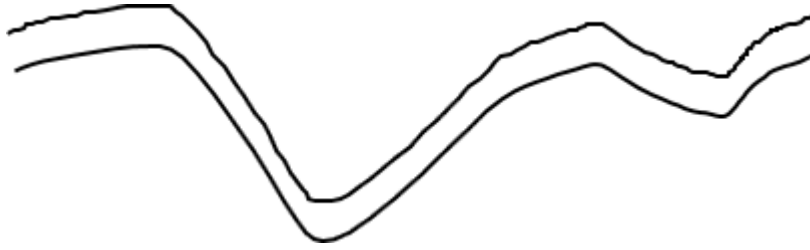


Figure 11-6. Smoothing in action

[Example 11-19](#) uses a mixture of LINQ to Objects and Rx's LINQ implementation. The query expression itself uses Rx, but the range variable, `points`, is of type `IList<Point>` (because `Buffer` returns an `IObservable<IList<Point>>` in this example). So the nested queries that invoke the `Average` operator on `points` will get the LINQ to Objects implementation.

If the `Buffer` operator's input is hot, it will produce a hot observable as a result. So you could subscribe to the observable in the `smoothed` variable in [Example 11-19](#) with similar code to the final line of [Example 11-14](#), and it would show the smoothed line in real time as you drag the mouse. As discussed, there will be a slight lag—the code specifies a skip of two, so it will update the screen only for every other mouse event. Averaging over the last five points will also increase the gap between the mouse pointer and the end of the line. With these parameters, the discrepancy is small enough not to be too distracting, but with more aggressive smoothing, it could get annoying.

Window versus Buffer

The `Window` operator is very similar to the `Buffer` operator, but instead of presenting each window as an `IList<T>`, it provides an `IObservable<T>`. If you used `Window` on `dragPositions` in [Example 11-19](#), the result would be `IObservable<IObservable<Point>>`. [Figure 11-7](#) shows how the `Window` operator would work in the last of the scenarios illustrated in [Figure 11-5](#), and as you can see, it can start each window sooner. It doesn't have to wait until all of the items in the window are available; instead of providing a fully populated list containing the win-

dow, each output item is an `IObservable<T>` that will produce the window's items as and when they become available. Each observable produced by `Window` completes immediately after supplying the final item (i.e., at the same instant at which `Buffer` would have provided the whole window). So, if your processing depends on having the whole window, `Window` can't get it to you any faster, because it's ultimately governed by the rate at which input items arrive, but it will start to provide values earlier.

One potentially surprising feature of the observables produced by `Window` in this example is their start times. Whereas they end immediately after producing their final item, they do not start immediately before producing their first. The observable representing the very first window starts right away—you will receive that observable as soon as you subscribe to the observable of observables the operator returns. So the first window will be available immediately, even if the `Window` operator's input hasn't done anything yet. Then each new window starts as soon as all the input items it needs to skip have been received. In this example, I'm using a skip count of one, so the second window starts after the input has produced one item, the third after two have been produced, and so on.

As you'll see later in this section, and also in [“Timed Sequences”](#), `Window` and `Buffer` support some other ways to define when each window starts and stops. The general pattern is that as soon as the `Window` operator gets to a point where a new item from the source would go into a new window, the operator creates that window, anticipating the window's first item rather than waiting for it (see [Figure 11-7](#)).

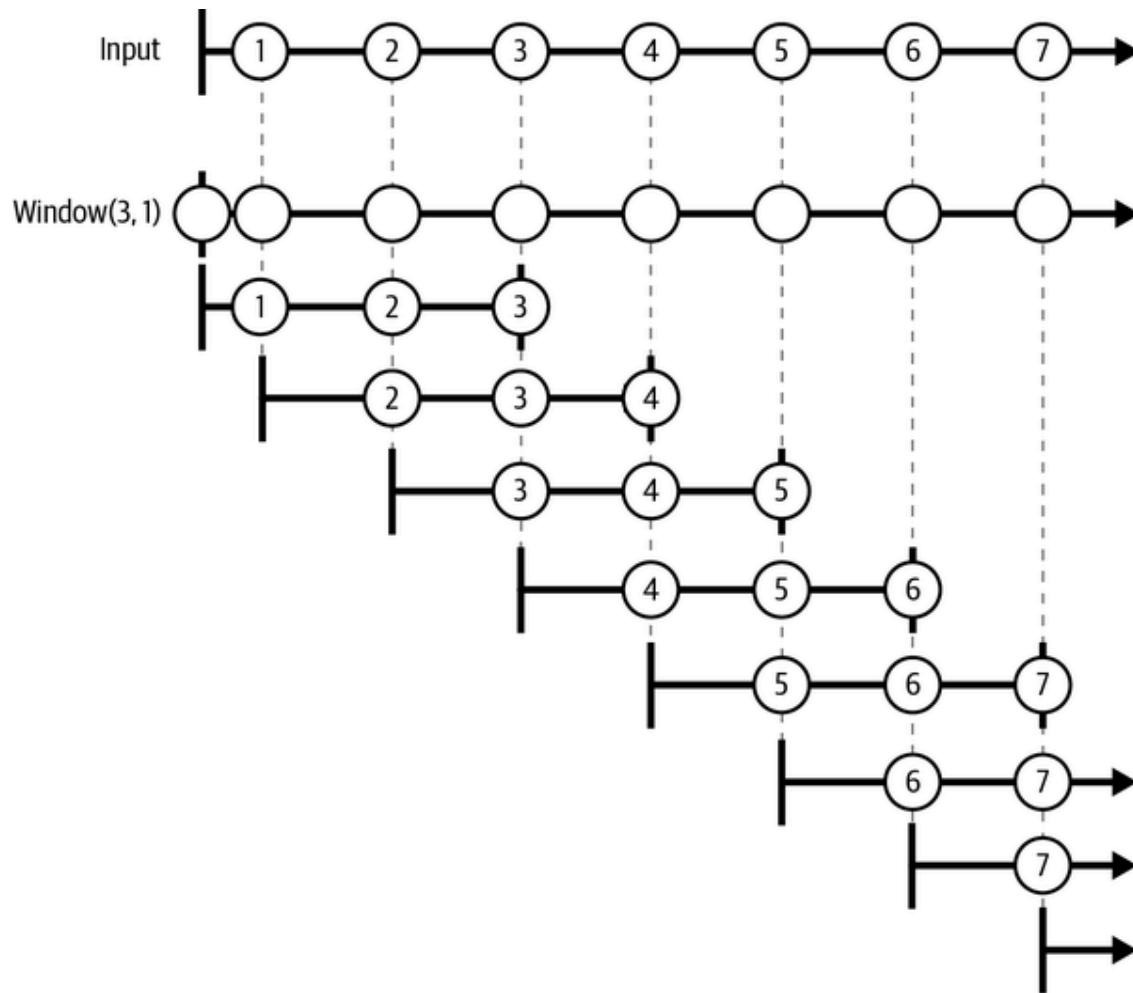


Figure 11-7. `Window` operator

NOTE

If the input completes, all currently open windows will also complete. This means that it's possible to see empty windows. (In fact, with a skip size of one, you're guaranteed to get one empty window if the source completes.) In [Figure 11-7](#), one window right at the bottom has started but has not yet produced any items. If the input were to complete without producing any more items, the three observable sources still in progress would also complete, including that final one that hasn't yet produced anything.

Because `Window` delivers items into windows as soon as the source provides them, it might enable you to get started with processing sooner than you can with `Buffer`, perhaps improving overall responsiveness. The downside of `Window` is that it tends to be more complex—your subscribers will start receiving output values before all the items for the corresponding input window are available. Whereas `Buffer` provides you with a list that you can inspect at your leisure, with `Window`, you'll need

to continue working in Rx's world of sequences that produce items only when they're good and ready. To perform the same smoothing as [Example 11-19](#) with `Window` requires the code in [Example 11-20](#).

Example 11-20. Smoothing with `Window`

```
IObservable<Point> smoothed =  
    from points in dragPositions.Window(5, 2)  
    from totals in points.Aggregate(  
        new { X = 0.0, Y = 0.0, Count = 0 },  
        (acc, point) => new  
            { X = acc.X + point.X, Y = acc.Y + point.Y, Count = acc.Count + 1 })  
    where totals.Count > 0  
    select new Point(totals.X / totals.Count, totals.Y / totals.Count);
```

This is more complicated because I've been unable to use the `Average` operator, due to the need to cope with the possibility of empty windows. (Strictly speaking, that doesn't matter in these examples where I have one `Polyline` that keeps getting longer and longer. But a real application would likely want to group the points by drag operation, to create a new line for each drag. Since each individual observable source of points would complete at the end of the drag, empty windows would be possible, and in general any use of `Window` will need to cope with that.) The `Average` operator produces an error if you provide it with an empty sequence, so I've used the `Aggregate` operator instead, which lets me add a `where` clause to filter out empty windows instead of crashing. But that's not the only aspect that is more complex.

As I mentioned earlier, all of Rx's aggregation operators—`Aggregate`, `Min`, `Max`, and so on—work differently than with most LINQ providers. LINQ requires these operators to reduce the stream down to a single value, so they normally return a single value. For example, if I were to call the LINQ to Objects version of `Aggregate` with the arguments shown in [Example 11-20](#), it would return a single value of the anonymous type I'm using for my accumulator. But in Rx, the return type is `IObservable<T>` (where `T` is that accumulator type in this case). It still produces a single value, but it presents that value through an observable source. Unlike LINQ to Objects, which can enumerate its input to calcu-

late, say, an average, the Rx operator has to wait for the source to provide its values, so it can't produce an aggregate of those values until the source says it has finished.

Because the `Aggregate` operator returns an `IObservable<T>`, I've had to use a second `from` clause. This passes that source to the `SelectMany` operator, which extracts all values and makes them appear in the final stream—in this case, there is just one value (per window), so `SelectMany` is effectively unwrapping the averaged point from its single-item stream.

The code in [Example 11-20](#) is more complex than [Example 11-19](#), and I think it's considerably harder to understand how it works. Worse, it doesn't even offer any benefit. The `Aggregate` operator will begin its work as soon as inputs become available, but the code cannot produce the final result—the average—until it has seen every point in the window. If I'm going to have to wait until the end of the window before I can update the UI, I may as well stick with `Buffer`.

So, in this particular case, `Window` was a lot more work for no benefit. However, if the work being done on the items in the window was less trivial, or if the volumes of data involved were so large that you didn't want to buffer the entire window before starting to process it, the extra complexity could be worth the benefit of being able to start the aggregation process without having to wait for the whole input window to become available.

Demarcating windows with observables

The `Window` and `Buffer` operators provide some other ways of defining when windows should start and finish. Just as the join operators can specify duration with an observable, you can supply a function that returns a duration-defining observable for each window. [Example 11-21](#) uses this to break keyboard input into words. The `keySource` variable in this example is the observable sequence from [Example 11-10](#) that produces an item for each keypress.

Example 11-21. Breaking text into words with windows

```
IObservable<IObservable<char>> wordWindows = keySource.Window(  
    () => keySource.FirstAsync(char.IsWhiteSpace));  
  
IObservable<string> words = from wordWindow in wordWindows  
    from chars in wordWindow.ToArray()  
    select new string(chars).Trim();  
  
words.Subscribe(word => Console.WriteLine("Word: " + word));
```

The `Window` operator will immediately create a new window in this example, and it will also invoke the lambda I've supplied to find out when that window should end. It will keep it open until the observable source the lambda returns either produces a value or completes. When that happens, `Window` will immediately open the next window, invoking the lambda again to get another observable to determine the length of the second window, and so on. The lambda here produces the next whitespace character from the keyboard, so the window will close on the next space. In other words, this breaks the input sequence into a series of windows where each window contains zero or more nonwhitespace characters followed by one whitespace character.

The observable sequence the `Window` operator returns presents each window as an `IObservable<char>`. The second statement in [Example 11-21](#) is a query that converts each window to a string. (This will produce empty strings if the input contains multiple adjacent whitespace characters. That's consistent with the behavior of the `string` type's `Split` method, which performs the pull-oriented equivalent of this partitioning. If you don't like it, you can always filter out the blanks with a `where` clause.)

Because [Example 11-21](#) uses `Window`, it will start making characters for each word available as soon as the user types them. But because my query calls `ToArray` on the window, it will end up waiting until the window completes before producing anything. This means `Buffer` would be equally effective. It would also be simpler. As [Example 11-22](#) shows, I don't need a second `from` clause to collect the completed window if I use

`Buffer`, because it provides me with windows only once they are complete.

Example 11-22. Word breaking with `Buffer`

```
IObservable<IList<char>> wordWindows = keySource.Buffer(  
    () => keySource.FirstAsync(char.IsWhiteSpace));  
  
IObservable<string> words = from wordWindow in wordWindows  
    select new string(wordWindow.ToArray()).Trim();
```

The Scan Operator

The `Scan` operator is very similar to the standard `Aggregate` operator, with one difference. Instead of producing a single result after its source completes, it produces a sequence containing each accumulator value in turn. To illustrate this, I will first introduce a record type that will act as a very simple model for a stock trade. This type, shown in [Example 11-23](#), also defines a static method that provides a randomly generated stream of trades for test purposes.

Example 11-23. Simple stock trade with test stream

```
public record Trade(string StockName, decimal UnitPrice, int Number)  
{  
    public static IObservable<Trade> GetTestStream()  
    {  
        return Observable.Create<Trade>(observer =>  
        {  
            string[] names = { "MSFT", "GOOGL", "AAPL" };  
            var r = new Random(0);  
            for (int i = 0; i < 100; ++i)  
            {  
                var t = new Trade(  
                    StockName: names[r.Next(names.Length)],  
                    UnitPrice: r.Next(1, 100),  
                    Number: r.Next(10, 1000));  
                observer.OnNext(t);  
            }  
        })  
    }  
}
```

```

        observer.OnCompleted();
        return Disposable.Empty;
    });
}
}

```

[Example 11-24](#) shows the normal `Aggregate` operator being used to calculate the total number of stocks traded, by adding up the `Number` property of every trade. (You'd normally just use the `Sum` operator, but I'm showing this for comparison with `Scan`.)

Example 11-24. Summing with `Aggregate`

```

IObservable<Trade> trades = Trade.GetTestStream();

IObservable<long> tradeVolume = trades.Aggregate(
    0L, (total, trade) => total + trade.Number);
tradeVolume.Subscribe(Console.WriteLine);

```

This displays a single number, because the observable produced by `Aggregate` provides only a single value. [Example 11-25](#) shows almost exactly the same code but using `Scan` instead.

Example 11-25. Running total with `Scan`

```

IObservable<Trade> trades = Trade.GetTestStream();

IObservable<long> tradeVolume = trades.Scan(
    0L, (total, trade) => total + trade.Number);
tradeVolume.Subscribe(Console.WriteLine);

```

Instead of producing a single output value, this produces one output item for each input, which is the running total for all items the source has produced so far. `Scan` is particularly useful if you need aggregation-like behavior in an endless stream, such as one based on an event source.

`Aggregate` is no use in that scenario because it will not produce anything if its input never completes.

The Amb Operator

Rx defines an operator with the somewhat cryptic name of `Amb`. (See the sidebar, [“Why Amb?”](#)) This takes any number of observable sequences and waits to see which one does something first. (The documentation talks about which of the inputs “reacts” first. This means that it calls any of the three `IObservable<T>` methods.) Whichever input jumps into action first effectively becomes the `Amb` operator’s output—it forwards everything the chosen stream does, immediately unsubscribing from the other streams. (If any of them manage to produce elements after the first stream does, but before the operator has had time to unsubscribe, those elements will be ignored.)

WHY AMB?

The `Amb` operator’s name is short for *ambiguous*. This seems like a violation of Microsoft’s own class library design guidelines, which forbid abbreviations unless the shortened form is more widely used than the full name and likely to be understood even by nonexperts. This operator’s name is well established—it was introduced in 1963 in a paper by John McCarthy (inventor of the LISP programming language). However, it’s not all that widely used, so the name fails the test of being instantly understandable by nonexperts.

However, the expanded name isn’t really any more transparent. If you’re not already familiar with the operator, the name `Ambiguous` wouldn’t be much more help in trying to guess what it does than just `Amb`. If you are familiar with it, you will already know that it’s called `Amb`. So there is no obvious downside to using the abbreviation, and there’s a benefit for people who already know it.

Another reason the Rx team used this name was to pay homage to John McCarthy, whose work was profoundly influential for computing in general, and for the LINQ and Rx projects in particular. (McCarthy’s work had a direct impact on many of the features discussed in this chapter and [Chapter 10](#).)

You might use this operator to optimize a system's response time by sending a request to multiple machines in a server pool and using the result from whichever responds first. (There are dangers with this technique, not least of which is that it could increase the overall load on your system so much that the effect is to slow everything down, including the operations you were hoping to speed up. Nevertheless, careful selective application of this technique can sometimes be successful.)

DistinctUntilChanged

The final operator I'm going to describe in this section is very simple but rather useful. The `DistinctUntilChanged` operator removes adjacent duplicates. Suppose you have an observable source that produces items on a regular basis but tends to produce the same value multiple times in a row. You might need to take action only when a different value emerges. `DistinctUntilChanged` is for exactly this scenario—when its input produces an item, it will be passed on only if it was different from the previous item (or if it was the first item).

I've not yet shown all of the Rx operators I want to introduce. However, the remaining ones, which I'll discuss in [“Timed Sequences”](#), are all time sensitive. And before I can show those, I need to describe how Rx handles timing.

Schedulers

Rx performs certain work through *schedulers*. A scheduler is an object that provides three services:

1. Deciding when to execute a particular piece of work. For example, when an observer subscribes to a cold source, should the source's items be delivered to the subscriber immediately, or should that work be deferred to reduce the risk of re-entrancy problems?
2. Running work in a particular context. A scheduler might decide always to execute work on a specific thread, for example.

3. Keeping track of time. Some Rx operations are time dependent; to ensure predictable behavior and to enable testing, schedulers provide a virtualized model for time, so Rx code does not have to depend on the current time of day reported by .NET's `DateTimeOffset` class.

The scheduler's first two roles are sometimes interdependent. For example, Rx supplies a few schedulers for use in UI applications. For example, there's `DispatcherScheduler` for WPF applications and `ControlScheduler` for Windows Forms programs. There's no specific support today for MAUI, but there is a more generic one called `SynchronizationContextScheduler`, which will work in all .NET UI frameworks, albeit with slightly less control over the details than the framework-specific ones. All of these have a common characteristic: they ensure that work executes in a suitable context for accessing UI objects, which typically means running the work on a particular thread. If code that schedules work is running on some other thread, the scheduler may have no choice but to defer the work, because it will not be able to run it until the UI framework is ready. This might mean waiting for a particular thread to finish whatever it is doing. In this case, running the work in the right context necessarily also has an impact on when the work is executed.

This isn't always the case, though. Rx provides two schedulers that use the current thread. One of them, `ImmediateScheduler`, is extremely simple: it runs work the instant it is scheduled. When you give this scheduler some work, it won't return until the work is complete. The other, `CurrentThreadScheduler`, maintains a work queue, which gives it some flexibility with ordering. For example, if some work is scheduled in the middle of executing some other piece of work, it can allow the work item in progress to finish before starting on the next. If no work items are queued or in progress, `CurrentThreadScheduler` runs work immediately, just like `ImmediateScheduler`. When a work item it has invoked completes, the `CurrentThreadScheduler` inspects the queue and will invoke the next item if it's not empty. So it attempts to complete all work items as quickly as possible, but unlike `ImmediateScheduler`, it will not start to process a new work item before the previous one has finished.

Specifying Schedulers

Rx operations often do not go through schedulers. Many observable sources invoke their subscribers' methods directly. Sources that can generate a large number of items in quick succession are typically an exception. For example, the `Range` and `Repeat` methods for creating sequences use a scheduler to govern the rate at which they provide items to new subscribers. You can pass in an explicit scheduler or let them pick a default one. You can also get a scheduler involved explicitly even when using sources that don't accept one as an argument.

ObserveOn

A common way to specify a scheduler is with one of the `ObserveOn` extension methods defined by various static classes in the `System.Reactive.Linq` namespace.⁴ This is useful if you want to handle events in a specific context (such as the UI thread) even though they may originate from somewhere else.

You can invoke `ObserveOn` on any `IObservable<T>`, passing in an `IScheduler`, and it returns another `IObservable<T>`. If you subscribe to the observable that returns, your observer's `OnNext`, `OnCompleted`, and `OnError` methods will all be invoked through the scheduler you specified. [Example 11-26](#) uses this to ensure that it's safe to update the UI in the item handler callback.

Example 11-26. `ObserveOn` specific scheduler

```
IObservable<Trade> trades = GetTradeStream();
IObservable<Trade> tradesInUiContext =
    trades.ObserveOn(DispatcherScheduler.Current);
tradesInUiContext.Subscribe(t =>
{
    tradeInfoTextBox.AppendText(
        $"{t.StockName}: {t.Number} at {t.UnitPrice}\r\n");
});
```

In this example, I used the `DispatcherScheduler` class's static `Current` property, which returns a scheduler that executes work via the current thread's `Dispatcher`. (`Dispatcher` is the class that manages the UI message loop in WPF applications.) Rx's `DispatcherObservable` class defines various extension methods providing WPF-specific overloads, and instead of passing in a scheduler, I can call `ObserveOn` passing just a `Dispatcher` object. I could use this in the codebehind for a UI element with code such as that in [Example 11-27](#).

Example 11-27. `ObserveOn` WPF `Dispatcher`

```
IObservable<Trade> tradesInUiContext = trades.ObserveOn(this.Dispatcher);
```

The advantage of this approach is that I don't need to be on the UI thread at the point at which I call `ObserveOn`. The `Current` property used in [Example 11-26](#) works only if you are on the thread for the dispatcher you require. If I'm already on that thread, there's an even simpler way to set this up. I can use the `ObserveOnDispatcher` extension method, which obtains a `DispatcherScheduler` for the current thread's dispatcher, as shown in [Example 11-28](#).

Example 11-28. Observing on the current dispatcher

```
IObservable<Trade> tradesInUiContext = trades.ObserveOnDispatcher();
```

SubscribeOn

Most of the various `ObserveOn` extension methods have corresponding `SubscribeOn` methods. (There's also `SubscribeOnDispatcher`, the counterpart of `ObserveOnDispatcher`.) Instead of arranging for each call to an observer's methods to be made through the scheduler, `SubscribeOn` performs the call to the source observable's `Subscribe` method through the scheduler. And if you unsubscribe by calling `Dispose`, that will also be delivered through the scheduler. This can be important for cold sources, because many perform significant work in their `Subscribe` method, some even delivering all of their items immediately.

NOTE

In general, there's no guarantee of any correspondence between the context in which you subscribe to a source and the context in which the items it produces will be delivered to a subscriber. Some sources will notify you from their subscription context, but many won't. If you need to receive notifications in a particular context, then unless the source provides some way to specify a scheduler, use `ObserveOn`.

Passing schedulers explicitly

Some operations accept a scheduler as an argument. You will tend to find this in operations that can generate many items. The `Observable.Range` method that generates a sequence of numbers optionally takes a scheduler as a final argument to control the context from which these numbers are generated. This also applies to the APIs for adapting other sources, such as `IEnumerable<T>` to observable sources, as described in [“Adaptation”](#).

Another scenario in which you can usually provide a scheduler is when using an observable that combines inputs. Earlier, you saw how the `Merge` operator combines the output of multiple sequences. You can provide a scheduler to tell the operator to subscribe to the sources from a specific context.

Finally, timed operations all depend on a scheduler. I will show some of these in [“Timed Sequences”](#).

Built-in Schedulers

I've already described UI-oriented schedulers such as `DispatcherScheduler` (for WPF), `ControlScheduler` (for Windows Forms), and `SynchronizationContextScheduler`, and also the two schedulers for running work on the current thread, `CurrentThreadScheduler` and `ImmediateScheduler`. But there are some others worth being aware of.

`EventLoopScheduler` runs all work items on a specific thread. It can create a new thread for you, or you can provide it with a callback method that it will invoke when it wants you to create the thread. You might use this in a UI application to process incoming data. It lets you move work off the UI thread to keep the application responsive but ensures that all processing happens on a single thread, which can simplify concurrency issues.

`NewThreadScheduler` creates a new thread for each top-level work item it processes. (If that work item spawns further work items, those will run on the same thread, rather than creating new ones.) This is appropriate only if you need to do a lot of work for each item, because threads have relatively high startup and teardown costs in Windows. You are normally better off using a thread pool if you need concurrent processing of work items.

`ThreadPoolScheduler` uses the Task Parallel Library's (TPL) thread pool. The TPL, described in [Chapter 16](#), provides an efficient pool of threads that can reuse a single thread for multiple work items, amortizing the startup costs of creating the thread.

`ThreadPoolScheduler` uses the CLR's thread pool to run work. This is similar in concept to the TPL thread pool, but it's a somewhat older piece of technology. (The TPL was introduced in .NET 4.0, but the CLR thread pool has existed since v1.0.) This is a bit less efficient in certain scenarios. Rx introduced this scheduler because early versions of Rx supported old versions of .NET that didn't have the TPL. It retains it for backward-compatibility reasons.

`HistoricalScheduler` is useful when you want to test time-sensitive code without needing to execute your tests in real time. All schedulers provide a time-keeping service, but the `HistoricalScheduler` lets you decide the exact rate at which you want the scheduler to behave as though time is elapsing. So, if you need to test what happens if you wait 30 seconds, you can just tell the `HistoricalScheduler` to act as though 30 seconds have passed, without having to actually wait.

Subjects

Rx defines various *subjects*, classes that implement both `IObserver<T>` and `IObservable<T>`. These can sometimes be useful if you need Rx to provide a robust implementation of either of these interfaces, but the usual `Observable.Create` or `Subscribe` methods are not convenient. For example, perhaps you need to provide an observable source, and there are several different places in your code from which you want to provide values for that source to produce. This is awkward to fit into the `Create` method's subscription callback model and can be easier to handle with a subject. Some of the subject types provide additional behavior, but I'll start with the simplest.

Subject<T>

`Subject<T>` relays calls to all observers that have subscribed using its `IObservable<T>` interface. So, if you subscribe one or more observables to a `Subject<T>` and then call `OnNext`, the subject will call `OnNext` on each of its subscribers. It's the same for the other methods, `OnCompleted` and `OnError`. This multicast relay behavior is very similar to the facility provided by the `Publish` operator⁵ I used in [Example 11-10](#), so `Subject<T>` provides an alternative way for me to remove all of the code for tracking subscribers from my `KeyWatcher` source, as [Example 11-29](#) shows.

Example 11-29. Implementing `IObservable<T>` with a `Subject<T>`

```
public class KeyWatcher
{
    private readonly Subject<char> _subject = new();

    public IObservable<char> Keys => _subject;

    public void Run()
    {
        while (true)
        {
            _subject.OnNext(Console.ReadKey(true).KeyChar);
        }
    }
}
```

```
    }  
  }  
}
```

This is much simpler than the original in [Example 11-7](#). The combination of `Observable.Create` and the `Publish` operator in [Example 11-10](#) is arguably simpler still, but `Subject<T>` does offer two advantages. First, it's easier to see when the loop that generates keypress notifications runs. [Example 11-10](#) behaves in a very similar way, but unless you're familiar with how `Publish` works, it is not obvious how. Second, if I wanted to, I could call `_subject.OnNext` from anywhere inside my `KeyWatcher` class, whereas [Example 11-10](#) can only produce items inside the callback function invoked by `Observable.Create`. As it happens, this example doesn't need that flexibility, but in scenarios that do, a `Subject<T>` is helpful.

BehaviorSubject<T>

`BehaviorSubject<T>` works almost exactly like `Subject<T>`, with one difference: it immediately notifies any observer that subscribes. If you have already completed the subject, it'll just call `OnComplete` immediately on any new subscribers. Otherwise, `BehaviorSubject<T>` remembers the last item it received and hands that out to new subscribers. When you construct a `BehaviorSubject<T>`, you have to supply an initial value that it will provide to new subscribers until the first call to `OnNext`.

`BehaviorSubject<T>` is like a variable: it has a value that you can retrieve at any time, and which might change. Being reactive, you subscribe to retrieve its value, and your observer will be notified of any further changes. `BehaviorSubject<T>` has a mix of hot and cold characteristics. It provides a value instantly to any subscriber, making it seem like a cold source, but it broadcasts new values to all current subscribers, more like a hot source.

ReplaySubject<T>

`ReplaySubject<T>` records the values it receives so that it can replay old items to each new subscriber. Once it has provided a particular sub-

subscriber with all recorded items, it transitions into more hot-like behavior for that subscriber, forwarding all new incoming items. So, in the long run, every subscriber to a `ReplaySubject<T>` will, by default, see every item that the `ReplaySubject<T>` receives from its source, regardless of how early or late that subscriber subscribed to the subject.

`ReplaySubject<T>` is like `BehaviorSubject<T>` but with a longer memory. In its default configuration, it will consume ever more memory for as long as it is subscribed to a source. However, you can limit this.

`ReplaySubject<T>` offers various constructor overloads specifying an upper limit on either the number of items to replay or the time for which it will hold on to items. Obviously, if you do this, new subscribers can no longer depend on getting all of the items previously received.

`AsyncSubject<T>`

`AsyncSubject<T>` remembers the final value it receives. If you subscribe to an `AsyncSubject<T>` before its source has completed, your observer receives nothing until the source completes. But once the source has completed, the `AsyncSubject<T>` acts as a cold source that provides a single value, unless the source completed without providing a value, in which case this subject will complete all new subscribers immediately.

Adaptation

Interesting and powerful though Rx is, it would not be much use if it existed in a vacuum. If you are working with asynchronous notifications, it's possible that they will be supplied by an API that does not support Rx. Although `IObservable<T>` and `IObserver<T>` have been around for a long time (since .NET 4.0, which was released in 2010), not every API that could support these interfaces does. Also, because Rx's fundamental abstraction is a sequence of items, there's a good chance that at some point you might need to convert between Rx's push-oriented `IObservable<T>` and the pull-oriented equivalents `IEnumerable<T>` and `IAsyncEnumerable<T>`. Rx provides ways to adapt these and other kinds

of sources into `IObservable<T>`, and in some cases, it can adapt in either direction.

`IEnumerable<T>` and `IAsyncEnumerable<T>`

Any `IEnumerable<T>` can easily be brought into the world of Rx thanks to the `ToObservable` extension methods. These are defined by the `Observable` static class in the `System.Reactive.Linq` namespace.

[Example 11-30](#) shows the simplest form, which takes no arguments.

Example 11-30. Converting an `IEnumerable<T>` to an `IObservable<T>`

```
public static void ShowAll(IEnumerable<string> source)
{
    IObservable<string> observableSource = source.ToObservable();
    observableSource.Subscribe(Console.WriteLine);
}
```

The `ToObservable` method itself does not enumerate its input—it just returns a wrapper that implements `IObservable<T>`. This wrapper is a cold source, and each time you subscribe an observer to it, only then does it iterate through the input, passing each item to the observer's `OnNext` method and calling `OnCompleted` at the end. If the source throws an exception, this adapter will call `OnError`. [Example 11-31](#) shows how `ToObservable` might work if it weren't for the fact that it needs to use a scheduler.

Example 11-31. How `ToObservable` might look without scheduler support

```
public static IObservable<T> MyToObservable<T>(this IEnumerable<T> input)
{
    return Observable.Create((IObserver<T> observer) =>
    {
        bool inObserver = false;
        try
        {
            foreach (T item in input)
            {
                observer.OnNext(item);
            }
            observer.OnCompleted();
        }
        catch { }
    });
}
```

```

        {
            inObserver = true;
            observer.OnNext(item);
            inObserver = false;
        }
        inObserver = true;
        observer.OnCompleted();
    }
    catch (Exception ex)
    {
        if (inObserver)
        {
            throw;
        }
        observer.OnError(ex);
    }
    return () => { };
});
}

```

This is not how it really works. (A full implementation would have been much harder to read, defeating the purpose of the example, which was to show the basic idea behind `ToObservable`.) The real method uses a scheduler to manage the iteration process, enabling subscription to occur asynchronously if required. It also supports stopping the work if the observer's subscription is canceled early. There's an overload that takes a single argument of type `IScheduler`, which lets you tell it to use a particular scheduler; if you don't provide one, it'll use `CurrentThreadScheduler`.

When it comes to going in the other direction—that is, when you have an `IObservable<T>`, but you would like to treat it as an `IEnumerable<T>`—you can call the `ToEnumerable` extension methods, also provided by the `Observable` class. [Example 11-32](#) wraps an `IObservable<string>` as an `IEnumerable<string>` so that it can iterate over the items in the source using an ordinary `foreach` loop.

Example 11-32. Using an `IObservable<T>` as an `IEnumerable<T>`

```
public static void ShowAll(IObservable<string> source)
{
    foreach (string s in source.ToEnumerable())
    {
        Console.WriteLine(s);
    }
}
```

The wrapper subscribes to the source on your behalf. If the source provides items faster than you can iterate over them, the wrapper will store the items in a queue so you can retrieve them at your leisure. If the source does not provide items as fast as you can retrieve them, the wrapper will just wait until items become available. You should be wary of `ToEnumerable` though, because if no items are available, it will block your thread until the source produces an item. This risks deadlock—if a thread is stuck inside `ToEnumerable` that could prevent whatever progress was required for the source to produce its next item.

The `IAsyncEnumerable<T>` interface provides the same model as `IEnumerable<T>` but in a way that enables efficient, nonblocking asynchronous operation using the techniques discussed in [Chapter 17](#). Rx offers a `ToObservable` extension method for this and also a `ToAsyncEnumerable` method extension method for `IObservable<T>`. These both come from the `AsyncEnumerable` class, and to use that you will need a reference to a separate NuGet package called `System.Linq.Async`.

.NET Events

Rx can wrap a .NET event as an `IObservable<T>` using the `Observable` class's static `FromEventPattern` method. Earlier, in [Example 11-16](#), I used a `FileSystemWatcher`, a class from the `System.IO` namespace that raises various events when files are added, deleted, renamed, or otherwise modified in a particular folder. I needed to bring its events into Rx's world of `IObservable<T>`.

[Example 11-33](#) uses the same technique as the first part of that example, which I glossed over last time. This code uses the `Observable.FromEventPattern` static method to produce an observable source representing the watcher's `Created` event.

Example 11-33. Wrapping an event in an `IObservable<T>`

```
string path = Environment.GetFolderPath(Environment.SpecialFolder.MyDocuments);
var w = new FileSystemWatcher(path);
IObservable<EventPattern<FileSystemEventArgs>> changes =
    Observable.FromEventPattern<FileSystemEventHandler, FileSystemEventArgs>(
        h => w.Changed += h, h => w.Changed -= h);
w.IncludeSubdirectories = true;
w.EnableRaisingEvents = true;

changes.Subscribe(evt => Console.WriteLine(evt.EventArgs.FullPath));
```

This is significantly more complicated than just subscribing to the event in the normal way shown in [Chapter 9](#), and this particular example gains nothing from it. However, one benefit of using Rx is that if you were writing a UI application, you could use `ObserveOn` with a suitable scheduler to ensure that your handler was always invoked on the right thread, regardless of which thread raised the event. Another benefit—and the usual reason for doing this—is that you can use any of Rx's query operators to process the events. (That's why the original [Example 11-16](#) did this.)

The element type of the observable source that [Example 11-33](#) produces is `EventPattern<FileSystemEventArgs>`. Rx defines the generic `EventPattern<T>` type specifically for representing the raising of an event where the event's delegate type conforms to the standard pattern described in [Chapter 9](#) (i.e., it takes two arguments, the first being of type `object`, representing the object that raised the event, and the second being some type derived from `EventArgs`, containing information about the event). `EventPattern<T>` has two properties, `Sender` and `EventArgs`, corresponding to the two arguments that an event handler would receive. In effect, this is an object that represents what would normally be a method call to an event handler.

`Observable.FromEventPattern` is somewhat fiddly to use: [Example 11-33](#) has had to pass in a pair of lambdas, one to subscribe from the event and one to unsubscribe. This is due to a shortcoming of events: you can't pass an event as an argument. This is one of the ways in which Rx improves on events—once you're in the world of Rx, event sources and subscribers are both represented as objects (implementing `IObservable<T>` and `IObserver<T>`, respectively), making it straightforward to pass them into methods as arguments. But that doesn't help us at the point where we're dealing with an event that's not yet in Rx's world.

There is an alternative overload that seems slightly simpler: instead of a pair of lambdas, you can pass just the name of the event. However, this forces Rx to use reflection (described in [Chapter 13](#)) to discover the event's type and locate its add and remove methods at runtime. This causes a couple of problems. First, it can prevent the use of ahead-of-time (AOT) compilation, because AOT depends on being able to work out what our code will do at compile time. Second, it means the compiler can't help you with types—if you attach handlers to a .NET event directly with a lambda, the compiler can determine the argument types from the event definition, and you'll get a compiler error if you try to use the delegate-based `Observable.FromEventPattern` with the wrong type arguments. But if you pass the event name as a string, the compiler doesn't know which event you're using, meaning it can't tell you about certain kinds of mistakes. So it's usually best to use the approach shown in [Example 11-33](#).

Asynchronous APIs

.NET supports various asynchronous patterns, which I'll be describing in detail in Chapters [16](#) and [17](#). The first to be introduced in .NET was the Asynchronous Programming Model (APM). However, this pattern is not supported directly by the new C# asynchronous language features, so most .NET APIs now use the TPL, and for older APIs the TPL offers adapters that can provide a task-based wrapper for an APM-based API. Rx can represent any TPL task as an observable source.

The basic model for all of .NET's asynchronous patterns is that you start some work that will eventually complete, optionally producing a result.

So it may seem odd to translate this into Rx, where the fundamental abstraction is a sequence of items, not a single result. In fact, one useful way to understand the difference between Rx and the TPL is that `IObservable<T>` is analogous to `IEnumerable<T>`, while `Task<T>` is analogous to a property of type `T`. Whereas with `IEnumerable<T>` and properties, the caller decides when to fetch information from the source, with `IObservable<T>` and `Task<T>`, the source provides the information when it's ready. The choice of which party decides when to provide information is separate from the question of whether the information is singular or a sequence of items. So a mapping between singular asynchronous APIs and `IObservable<T>` seems a little mismatched. But then we can cross similar boundaries in the nonasynchronous world—LINQ defines various standard operators that produce a single item from a sequence, such as `First` or `Last`. Rx supports those operators, but it additionally supports going in the other direction: bringing singular asynchronous sources into a stream-like world. The upshot is an `IObservable<T>` source that produces just a single item (or reports an error if the operation fails). The analogy in the nonasynchronous world would be taking a single value and wrapping it in an array so that you can pass it to an API that requires an `IEnumerable<T>`.

[Example 11-34](#) uses this facility to produce an `IObservable<string>` that will either produce a single value containing the text downloaded from a particular URL or report a failure should the download fail.

Example 11-34. Wrapping a `Task<T>` as an `IObservable<T>`

```
public static IObservable<string> GetWebPageAsObservable(
    Uri pageUrl, IHttpConnectionFactory cf)
{
    async Task<string> GetPageAsync()
    {
        using HttpClient web = cf.CreateClient();
        return await web.GetStringAsync(pageUrl).ConfigureAwait(false);
    }
    return GetPageAsync().ToObservable();
}
```

The `ToObservable` method used in this example is an extension method defined for `Task` by Rx. For this to be available, you'll need the `System.Reactive.Threading.Tasks` namespace to be in scope.

One potentially unsatisfactory feature of [Example 11-34](#) is that it will attempt the download only once, no matter how many observers subscribe to the source. Depending on your requirements, that might be fine, but in some scenarios, it might make sense to attempt to download a fresh copy every time. If you want that, you should use the `Observable.FromAsync` method, because you pass that a lambda that it invokes each time a new observer subscribes. Your lambda returns a task that will then be wrapped as an observable source. [Example 11-35](#) uses this to start a new download for each subscriber.

Example 11-35. Creating a new task for each subscriber

```
public static IObservable<string> GetWebPageAsObservable(
    Uri pageUrl, IHttpClientFactory cf)
{
    return Observable.FromAsync(async () =>
    {
        using HttpClient web = cf.CreateClient();
        return await web.GetStringAsync(pageUrl);
    });
}
```

This might be suboptimal if you have many subscribers. On the other hand, it's more efficient when nothing attempts to subscribe at all.

[Example 11-34](#) starts the asynchronous work immediately without even waiting for any subscribers. That may be a good thing—if the stream will definitely have subscribers, kicking off slow work without waiting for the first subscriber will reduce your overall latency. However, if you are writing a class in a library that presents multiple observable sources, which might not all be used, deferring work until the first subscription might be better.

Timed Sequences

Because Rx can work with live streams of information, you may need to handle items in a time-sensitive way. For example, the rate at which items arrive might be important, or you may wish to group items based on when they were provided. In this final section, I'll describe some of the time-based operators that Rx offers.

Since schedulers play a central role with how Rx handles timing, all of the methods and operators described in this section offer overloads enabling you to specify the `IScheduler` they should use.

Timed Sources

The methods described in this section do not require an existing `IObservable<T>` as input. They create new `IObservable<long>` sequences from scratch.

Observable.Interval

Regularly produces values at the interval specified by an argument of type `TimeSpan`. The items are of type `long`. It produces values of zero, one, two, and so on. `Interval` handles each subscriber independently (i.e., it is a cold source), so although each subscriber will receive items at the same interval, they won't generally receive them at the same time.

Observable.Timer

The simplest overload waits for the duration specified with a `TimeSpan` argument then produces a single item. There are also overloads that accept an extra `TimeSpan`, which will repeatedly produce the value just like `Interval`. In fact, `Interval` is just a wrapper for `Timer`, offering a simpler API.

Timed Operators

The operators described in this section are all extension methods, taking an existing observable sequence as input and returning an `IObservable<T>` based on the input:

Timestamp

Reports the time at which each element entered the operator. This can be useful in cases where there may be a significant delay in between the item being produced and your code getting to handle it. (For example, if you have used `ObserveOn` to ensure that your handler always runs on the UI thread, delays occur when the UI thread may be busy.) [Example 11-36](#) uses this to show the times at which an `Interval` produces its items. As you can see, this turns the `IObservable<long>` returned by `Interval` into a sequence of `Timestamped<long>` elements. `Timestamped<T>` defines two properties, `Value` and `Timestamp`, providing the input element and the time it arrived at this operator, respectively.

Example 11-36. Timestamped items

```
IObservable<Timestamped<long>> src =  
    Observable.Interval(TimeSpan.FromSeconds(1)).Timestamp();  
src.Subscribe(i => Console.WriteLine(  
    $"Event {i.Value} at {i.Timestamp.ToLocalTime():T}"));
```

TimeInterval

Whereas `Timestamp` records the current time at which items are produced, its relative counterpart `TimeInterval` records the time between successive items. Given an `IObservable<T>`, this returns an `IObservable<TimeInterval<T>>`.

Throttle

Lets you limit the rate at which you process items. You pass a `TimeSpan` that specifies the minimum time interval you want between any two items. If the underlying source produces items

faster than this, `Throttle` will just discard them. If the source is slower than the specified rate, `Throttle` just passes everything straight through. Surprisingly (or at least, I found this surprising), once the source exceeds the specified rate, `Throttle` drops *everything* until the rate drops back down below the specified level. So, if you specify a rate of 10 items a second, and the source produces 100 per second, it won't simply return every 10th item—it'll return nothing until the source slows down.

Sample

Produces items from its input at the interval specified by its `TimeSpan` argument, regardless of the rate at which the input observable is generating items. If the underlying source produces items faster than the chosen rate, `Sample` drops items to limit the rate. However, if the source is running slower, the `Sample` operator will just repeat the last value to ensure a constant supply of notifications.

Timeout

Passes everything through from its source observable unless the source leaves too large a gap either between the subscription time and the first item or between two subsequent calls to the observer. You specify the minimum acceptable gap with a `TimeSpan` argument. If no activity occurs within that time, the `Timeout` operator completes by reporting a `TimeoutException` to `OnError`.

Delay

Time-shifts an observable source. You can pass a `TimeSpan`, in which case the operator will delay everything by the specified amount, or you can pass a `DateTimeOffset`, indicating a specific time at which you would like it to start replaying its input. Alternatively, you can pass an observable, and whenever that observable first produces something or completes, the `Delay` operator will start producing the values it has stored. The `Delay` operator attempts to maintain the same spacing between inputs. So, if the underlying source produces an item immediately, then another

item after three seconds, and then a third item after a minute, the observable produced by `Delay` will produce items separated by the same time intervals.

The fidelity with which `Delay` can reproduce the exact timing of the items is determined by the nature of the scheduler you're using and the available CPU capacity on the machine. For example, if you use one of the UI-based schedulers, it will be limited by the availability of the UI thread and the rate at which that can dispatch work.

`DelaySubscription`

Time-shifts subscription to an observable source.

`DelaySubscription` offers a similar set of overloads to the `Delay` operator, but the way it tries to effect a delay is different. When you subscribe to an observable source produced by `Delay`, it will immediately subscribe to the underlying source and start buffering items, forwarding each item only when the required delay has elapsed.

The strategy employed by `DelaySubscription` is simply to delay the subscription to the underlying source and then forward each item immediately. This typically works well for cold sources, because with those, delaying the start of work will typically time-shift the entire process. But for a hot source, `DelaySubscription` will cause you to miss any events that occurred during the delay, and after that, you'll start getting events with no time shift. So `Delay` is more dependable—by time-shifting each item individually, it works for both hot and cold sources. However, it has to do more work—it needs to buffer everything it receives for the delay duration. For busy sources or long delays, this could consume a lot of memory. And the attempt to reproduce the original timings with a time shift is considerably more complicated than just passing items straight on. So, in scenarios where it is viable, `DelaySubscription` is more efficient.

Timed Windowing Operators

I described the `Buffer` and `Window` operators earlier, but I didn't show their time-based overloads. As well as being able to specify a window size and skip count, or to mark window boundaries with an ancillary observable source, you can also specify time-based windows.

If you pass just a `TimeSpan`, both operators will break the input into adjacent windows at the specified interval. [Example 11-37](#) applies `Buffer` this way to the `words` observable defined in [Example 11-22](#) to estimate the words per minute.

Example 11-37. Timed windows with `Buffer`

```
IObservable<int> wordGroupCounts =  
    from wordGroup in words.Buffer(TimeSpan.FromSeconds(6))  
    select wordGroup.Count * 10;  
wordGroupCounts.Subscribe(c => Console.WriteLine($"Words per minute: {c}"));
```

There are also overloads accepting both a `TimeSpan` and an `int`, enabling you to close the current window (thus starting the next window) either when the specified interval elapses or when the number of items exceeds a threshold. In addition, there are overloads accepting two `TimeSpan` arguments. These support the time-based equivalent of the combination of a window size and a skip count. The first `TimeSpan` argument specifies the window duration, while the second specifies the interval at which to start new windows. This means the windows do not need to be strictly adjacent—you can have gaps between them, or they can overlap. [Example 11-38](#) uses this to provide more frequent estimates of the word rate while still using a six-second window.

Example 11-38. Overlapping timed windows

```
IObservable<int> wordGroupCounts =  
    from wordGroup in words.Buffer(TimeSpan.FromSeconds(6),  
                                    TimeSpan.FromSeconds(1))  
    select wordGroup.Count * 10;
```

Reactor—Rx as a Service

Although Rx is now a fully community-supported project,⁶ it was originally created by Microsoft. The same team also produced a set of components that makes it possible to host long-running Rx queries in a service. Microsoft has been using this internally for years to provide event-driven functionality in a variety of its online services, including the Bing search engine and the online versions of Office. It enables features such as setting up alerts that tell you when you'll need to leave to get to an appointment on time given current traffic conditions, for example. It has a proven track record of being able to maintain millions of active queries. For many years this was an internal project, but it is now an open source project called Reactor. The code for the core libraries that make this possible is hosted at [the Reactor source repository](#), and there is [a site with documentation and supporting information](#).⁷

Reactor takes the programming model of Rx—observable sequences, subjects, and operators—and exploits .NET's expression tree features described in [Chapter 9](#) to enable queries to be stored or sent across the network. It also provides versions of standard LINQ operators that are able to persist their state, enabling queries with stateful operators (e.g., `Aggregate`, `DistinctUntilChanged`, or anything else that needs to remember something about what it has already seen) to survive beyond the lifetime of any single process. This enables an application to define a LINQ query to some observable source of data and set up a subscription to that query that will be hosted in a server pool, persisting with an arbitrarily long lifetime. Reactor is designed to offer the same kind of durability as a database, so some of Microsoft's applications have Rx queries that have been running uninterrupted for several years.

The relationship between Rx and Reactor is not unlike the relationship between LINQ to Objects and Entity Framework (EF) Core. As you saw in [Chapter 10](#), LINQ to Objects is built on `IEnumerable<T>`, and it works entirely in-memory, with no persistence or cross-process capability. EF Core takes the same basic concepts and offers most of the same operators, but by building on the expression-tree-based `IQueryable<T>`, EF Core is able

to send representations of an application's queries over to a database server so that they can be executed remotely—EF Core brings LINQ into a world of durable persistence and distributed execution. Similarly, whereas Rx is built on `IObservable<T>` and runs entirely in-memory, Reaqtor uses an expression-tree-based interface `IQueryable<T>`. (Note the Q instead of an O, denoting its similarity in concept to `IQueryable<T>`.) `IQueryable<T>` looks very similar to `IObservable<T>` and offers all of the same operators, but because it works in the world of expression trees, it is possible for Reaqtor to convert queries into a form that can be sent over the network to a server farm, which can then reconstitute runnable versions of those queries hosted inside the server farm. It exploits the serializability to store the queries, enabling them to be migrated from one machine to another within the server farm, providing persistence and durability in the face of individual server failures. Reaqtor brings Rx into a world of durable persistence and distributed execution.

At the time of writing, there isn't an off-the-shelf hosted version of Reaqtor freely available, so it takes quite a lot of work to build something real from the Reaqtor libraries. But I've built a couple of applications on top of this with my employer, so I can say with confidence that it is certainly possible.

Summary

As you've now seen, the Reactive Extensions for .NET provide a lot of functionality. The concept underpinning Rx is a well-defined abstraction for sequences of items where the source decides when to provide each item, and a related abstraction representing a subscriber to such a sequence. By representing both concepts as objects, event sources and subscribers both become first-class entities, meaning you can pass them as arguments, store them in fields, and generally do anything with them that you can do with any other data type in .NET. While you can do all of that with a delegate too, .NET events are not first class. Moreover, Rx provides a clearly defined mechanism for notifying a subscriber of errors, something that neither delegates nor events handle well. As well as defining a

first-class representation for event sources, Rx defines a comprehensive LINQ implementation, which is why Rx is sometimes described as LINQ to Events. In fact, it goes well beyond the set of standard LINQ operators, adding numerous operators that exploit and help to manage the live and potentially time-sensitive world that event-driven systems occupy. Rx also provides various services for bridging between its basic abstractions and those of other worlds, including standard .NET events, `IEnumerable<T>`, and various asynchronous models.

- 1 You can download the full WPF example to which this snippet belongs as part of the [examples for this book](#).
- 2 It is missing the `OrderBy` and `ThenBy` operators, because these make little sense in a push-based world. They cannot produce any items until they have seen all of their input items.
- 3 Like some developers.
- 4 The overloads are spread across multiple classes because some of these extension methods are technology specific. WPF gets `ObserveOn` overloads that work directly with its `Dispatcher` class instead of `IScheduler`, for example.
- 5 In fact, `Publish` uses `Subject<T>` internally in the current version of Rx.
- 6 I became the primary Rx project maintainer in January 2023 by the way.
- 7 I am the primary maintainer for Reaqtor too.